

Notacja asymptotyczna. Algorytmy dziel i zwyciężaj – Wykład

4 marca 2026

1 Notacja asymptotyczna

$f, g: \mathbb{N} \rightarrow \mathbb{R}_+$ (rosnąca).

- $g(n) = O(f(n))$ jeśli $\exists c > 0 \exists n_0 \forall n \geq n_0 \quad g(n) \leq c \cdot f(n)$
- $g(n) = \Omega(f(n))$ jeśli $\exists c > 0 \exists n_0 \forall n \geq n_0 \quad c \cdot f(n) \leq g(n)$
- $g(n) = \Theta(f(n))$ jeśli $\exists c_1, c_2 > 0 \exists n_0 \forall n \geq n_0 \quad c_1 f(n) \leq g(n) \leq c_2 f(n)$

Przykłady:

$$\begin{array}{ll} n^a = O(n^b), & 0 < a \leq b \\ \log n = O(n^\varepsilon), & \varepsilon > 0 \\ a^n = O(b^n), & 1 < a \leq b \\ n^a = O(b^n), & a > 0, b > 1 \\ \log_a n = O(\log_b n), & a, b > 1 \end{array}$$

Przykład: $T(n) = 5n^3 + 2n^3 + 100n \cdot \log n \approx 5n^3 = \Theta(n^3)$.

Sortowanie = $O(n \log n)$ – merge sort.

2 Procedura rekurencyjna

Przykład 1 (Silnia). $n!$:

$$\begin{cases} 0! = 1 \\ n! = n \cdot (n-1)! \quad \text{dla } n \geq 1 \end{cases}$$

$n! = n \cdot (n-1) \cdot \dots \cdot 1$.

fact(n):

1. **if** $n = 0$ **return** 1
2. **return** $n \cdot \text{fact}(n-1)$

Przykład 2 (Ciąg Fibonacciego). F_n : $F_0 = 0$, $F_1 = 1$, $F_n = F_{n-1} + F_{n-2}$ dla $n \geq 2$.

1. **if** $n = 0$ **return** 0; **if** $n = 1$ **return** 1
2. **return** $\text{Fib}(n-1) + \text{Fib}(n-2)$

Wywołanie rekurencyjne dla tego samego problemu – nie dla podobnego, a tego samego – często dla mniejszych problemów.

3 Algorytmy dziel i zwyciężaj

Divide & conquer.

Algorytm typu dziel i zwyciężaj składa się z 3 części:

1. **Dziel** – problem jest dzielony na mniejsze podproblemy.
2. **Zwyciężaj** – rekurencyjnie rozwiązujemy te podproblemy.
3. **Połącz** – budujemy rozwiązanie całego problemu z mniejszych instancji.

Zazwyczaj zakładamy, że:

1. W fazie dziel problem jest dzielony na conajmniej dwa podproblemy.
2. Zakładamy, że te podproblemy są rozłączne (nie tak jak w rekurencji wyżej).

3.1 Mnożenie liczb n -cyfrowych

$x, y \in \mathbb{N}$ – n -cyfrowe.

Aby policzyć $x \cdot y$ musimy wykonać $O(n^2)$ mnożeń cyfr.

→ Znajdźmy lepsze rozwiązanie.

Dzielimy x i y na “dwie połowy”:

$$x = x_1x_2 \dots x_n = \underbrace{x_1x_2 \dots x_{n/2}}_{x_L} \underbrace{x_{n/2+1} \dots x_n}_{x_R} = 10^{n/2}x_L + x_R$$

$$y = 10^{n/2}y_L + y_R$$

$$x \cdot y = (10^{n/2}x_L + x_R)(10^{n/2}y_L + y_R) = 10^n x_L y_L + 10^{n/2}(x_L y_R + x_R y_L) + x_R y_R$$

$T(n)$ – złożoność czasowa tego algorytmu.

Podział x i y zajął nam $O(n)$ czasu. Dodawanie n -cyfrowych liczb $O(n)$. Mnożenie przez 10^n jest równoważne dopisaniu n zer, więc też zajmuje $O(n)$.

$$T(n) = 4 \cdot T\left(\frac{n}{2}\right) + O(n)$$

→ Można lepiej to zrobić:

$$x_L y_R + x_R y_L = (x_L - x_R) \cdot (y_R - y_L) + x_L y_L + x_R y_R$$

W tym rozwiązaniu zamiast 4 musimy policzyć tylko 3 iloczyny: $x_L y_L$, $x_R y_R$, $(x_L - x_R)(y_L - y_R)$.

Zatem

$$T(n) = 3 \cdot T\left(\frac{n}{2}\right) + O(n)$$

4 Uniwersalne twierdzenie o rekurencji (Master Theorem)

Twierdzenie 1 (Uniwersalne twierdzenie o rekurencji). Niech $a \geq 1$, $b > 1$ będą stałymi, $f(n)$ funkcją i

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

(gdzie $\frac{n}{b}$ znamy $\lfloor \frac{n}{b} \rfloor$ lub $\lceil \frac{n}{b} \rceil$) – ten ciąg podzieliśmy na a problemów $\frac{n}{b}$.

Wtedy

$$T(n) = \begin{cases} O(n^{\log_b a}) & \text{jeśli } f(n) = O(n^{\log_b a - \varepsilon}), \varepsilon > 0 \\ \Theta(n^{\log_b a} \cdot \log n) & \text{jeśli } f(n) = \Theta(n^{\log_b a}) \\ \Theta(f(n)) & \text{jeśli } f(n) = \Omega(n^{\log_b a + \varepsilon}), \varepsilon > 0, \quad a \cdot f\left(\frac{n}{b}\right) \leq c \cdot f(n), \quad c < 1 \text{ dla dużych } n \end{cases}$$

Zanim udowodnimy to, udowodnimy lemat.

Lemat 1. Niech $a \geq 1$, $b > 1$ będą stałymi i niech $f(n)$ będzie nieujemną funkcją dla potęgi b .
Jeśli

$$T(n) = \begin{cases} \Theta(1) & n = 1 \\ a \cdot T\left(\frac{n}{b}\right) + f(n) & \text{dla } n = b^i, i \geq 1 \end{cases}$$

to

$$T(n) = \Theta\left(n^{\log_b a}\right) + \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right)$$

Dowód.

$$\begin{aligned} T(n) &= f(n) + a \cdot T\left(\frac{n}{b}\right) = f(n) + a\left(f\left(\frac{n}{b}\right) + a \cdot T\left(\frac{n}{b^2}\right)\right) = \dots \\ &= f(n) + a f\left(\frac{n}{b}\right) + a^2 \cdot T\left(\frac{n}{b^2}\right) = \dots \\ &= f(n) + a f\left(\frac{n}{b}\right) + a^2 f\left(\frac{n}{b^2}\right) + a^3 \cdot T\left(\frac{n}{b^3}\right) = \dots \\ &= f(n) + a f\left(\frac{n}{b}\right) + \dots + a^{\log_b n - 1} f\left(\frac{n}{b^{\log_b n - 1}}\right) + a^{\log_b n} \cdot T(1) \end{aligned}$$

Ponieważ $a^{\log_b n} = b^{\log_b n \cdot \log_b a} = b^{\log_b n \cdot \log_n a \cdot \log_b n / \log_b n} = \dots = n^{\log_b a}$, stąd:

$$a^{\log_b n} \cdot T(1) = n^{\log_b a} \cdot T(1) = \Theta\left(n^{\log_b a}\right)$$

więc

$$T(n) = \Theta\left(n^{\log_b a}\right) + \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right)$$

□

4.1 Dowód twierdzenia (dla n będącego potęgą b)

Reszta też prawdziwa, ale dowód trudniejszy.

Rozważamy 3 przypadki:

Przypadek 1. $f(n) = O(n^{\log_b a - \varepsilon})$.

Wtedy $f\left(\frac{n}{b^i}\right) = O\left(\left(\frac{n}{b^i}\right)^{\log_b a - \varepsilon}\right)$.

$$\begin{aligned} \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right) &= \sum_{j=0}^{\log_b n - 1} a^j O\left(\left(\frac{n}{b^j}\right)^{\log_b a - \varepsilon}\right) = O\left(\sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j}\right)^{\log_b a - \varepsilon}\right) \\ &= n^{\log_b a - \varepsilon} \sum_{j=0}^{\log_b n - 1} \frac{a^j}{b^{j(\log_b a - \varepsilon)}} = n^{\log_b a - \varepsilon} \sum_{j=0}^{\log_b n - 1} \frac{b^{j\varepsilon}}{b^{j \log_b a}} \cdot a^j \\ &= n^{\log_b a - \varepsilon} \sum_{j=0}^{\log_b n - 1} \left(\frac{a \cdot b^\varepsilon}{b^{\log_b a}}\right)^j = n^{\log_b a - \varepsilon} \sum_{j=0}^{\log_b n - 1} (b^\varepsilon)^j \end{aligned}$$

(skończony szereg geometryczny)

$$= n^{\log_b a - \varepsilon} \cdot \frac{b^{\varepsilon \log_b n} - 1}{b^\varepsilon - 1} = n^{\log_b a - \varepsilon} \cdot \frac{n^\varepsilon - 1}{b^\varepsilon - 1} = n^{\log_b a - \varepsilon} \cdot O(n^\varepsilon) = O\left(n^{\log_b a}\right)$$

Korzystając z lematu:

$$T(n) = \sum a^j f\left(\frac{n}{b^j}\right) + \Theta\left(n^{\log_b a}\right) = O\left(n^{\log_b a}\right) + \Theta\left(n^{\log_b a}\right) = O\left(n^{\log_b a}\right)$$

Przypadek 2. $f(n) = \Theta(n^{\log_b a})$.

Wtedy $f(\frac{n}{b^i}) = \Theta((\frac{n}{b^i})^{\log_b a})$.

$$\sum_{j=0}^{\log_b n - 1} a^j f(\frac{n}{b^j}) = \sum_{j=0}^{\log_b n - 1} a^j \Theta((\frac{n}{b^j})^{\log_b a}) \stackrel{\text{podobne obliczenia}}{=} \Theta(n^{\log_b n \log_b a})$$

Korzystamy z lematu:

$$T(n) = \sum a^j f(\frac{n}{b^j}) + \Theta(n^{\log_b a}) = \Theta(n^{\log_b a} \cdot \log n) + \Theta(n^{\log_b a}) = \Theta(n^{\log_b a} \log n)$$

Przypadek 3. $f(n) = \Omega(n^{\log_b a + \varepsilon})$ i $a \cdot f(\frac{n}{b}) \leq c \cdot f(n)$ dla $c < 1$ i dużych n .

$f(\frac{n}{b}) \leq \frac{c}{a} f(n)$, więc $f(\frac{n}{b^i}) \leq (\frac{c}{a})^i \cdot f(n) = \frac{c^i}{a^i} f(n)$.

$$\sum_{j=0}^{\log_b n - 1} a^j f(\frac{n}{b^j}) \leq \sum_{j=0}^{\log_b n - 1} a^j \cdot \frac{c^j}{a^j} f(n) + O(1) = f(n) \cdot \sum c^j + O(1) \leq f(n) \cdot \sum_{j=0}^{\infty} c^j + O(1)$$

(nieskończony ciąg geometryczny, $c < 1$)

$$= f(n) \cdot \frac{1}{1-c} + O(1) = O(f(n))$$

Korzystamy z lematu:

$$\sum_{j=0}^{\log_b n - 1} a^j f(\frac{n}{b^j}) = a^0 f(\frac{n}{b^0}) + \sum_{j=1}^{\log_b n - 1} a^j f(\frac{n}{b^j}) = f(n) + (\geq 0) = \Omega(f(n))$$

$$T(n) = \sum a^j f(\frac{n}{b^j}) + O(n^{\log_b a}) = O(f(n)) + \Theta(n^{\log_b a}) = \Theta(f(n))$$

Wniosek 1. Niech $a \geq 1$, $b > 1$ będą stałymi, $a, f(n)$ funkcją taką, że $f(n) = \Theta(n^k)$.

Niech $T(n) = a \cdot T(\frac{n}{b}) + f(n)$. Wtedy

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & \text{jeśli } a > b^k \iff \log_b a > k \\ \Theta(n^{\log_b a} \log n) & \text{jeśli } a = b^k \iff \log_b a = k \\ \Theta(n^k) & \text{jeśli } a < b^k \iff \log_b a < k \end{cases}$$

4.2 Powrót do mnożenia liczb n -cyfrowych

Drugi algorytm:

$$T(n) = 4 \cdot T(\frac{n}{2}) + O(n), \quad a = 4, b = 2, k = 1, \quad 4 > 2^1 \implies T(n) = \Theta(n^{\log_2 4}) = n^2$$

Trzeci algorytm:

$$T(n) = 3 \cdot T(\frac{n}{2}) + O(n), \quad a = 3, b = 2, k = 1, \quad 3 > 2^1 \implies T(n) = \Theta(n^{\log_2 3}) = n^{1,59}$$

Mnożenie macierzy. Znajdowanie pary najbliższych punktów – Wykład

11 marca 2026

1 Mnożenie macierzy

A, B – macierze $n \times n$. $A \cdot B = C = ?$

Najpopularniejszy sposób mnożenia dwóch macierzy $n \times n$ działa w czasie $O(n^3)$. Musimy policzyć $O(n^2)$ wartości, obliczenie każdej zajmuje $O(n)$ czasu.

1.1 Algorytm Strassena

Zakładamy, że n jest potęgą 2 dla przejrzystości.

Dzielimy A i B na 4 macierze $\frac{n}{2} \times \frac{n}{2}$:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$
$$A \cdot B = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \cdot \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = C$$

gdzie:

$$C_{11} = A_{11} \cdot B_{11} + A_{12} \cdot B_{21}$$

$$C_{12} = A_{11} \cdot B_{12} + A_{12} \cdot B_{22}$$

$$C_{21} = A_{21} \cdot B_{11} + A_{22} \cdot B_{21}$$

$$C_{22} = A_{21} \cdot B_{12} + A_{22} \cdot B_{22}$$

Zamieniamy 1 mnożenie macierzy $n \times n$ na 8 mnożeń macierzy $\frac{n}{2} \times \frac{n}{2}$ + podział i dodawanie $\Theta(n^2)$.

Stąd mamy zależność rekurencyjną:

$$T(n) = 8 \cdot T\left(\frac{n}{2}\right) + O(n^2), \quad a = 8, \quad b = 2, \quad k = 2, \quad 8 > 2^2$$

$$T(n) = \Theta\left(n^{\log_2 8}\right) = \Theta(n^3)$$

Strassen zauważył, że jeśli obliczymy:

$$M_1 = (A_{12} - A_{22})(B_{21} + B_{22})$$

$$M_2 = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$M_3 = (A_{11} - A_{21})(B_{11} + B_{12})$$

$$M_4 = (A_{11} + A_{12}) \cdot B_{22}$$

$$M_5 = A_{11} \cdot (B_{12} - B_{22})$$

$$M_6 = A_{22} \cdot (B_{21} - B_{11})$$

$$M_7 = (A_{21} + A_{22}) \cdot B_{11}$$

to

$$C_{11} = M_1 + M_2 - M_4 + M_6$$

$$C_{12} = M_4 + M_5$$

$$C_{21} = M_6 + M_7$$

$$C_{22} = M_2 - M_3 + M_5 - M_7$$

Zatem

$$T(n) = 7 \cdot T\left(\frac{n}{2}\right) + \Theta(n^2), \quad a = 7, \quad b = 2, \quad k = 2, \quad 7 > 2^2$$

$$T(n) = \Theta\left(n^{\log_2 7}\right) = \Theta(n^{2,81})$$

1.2 Własności optymalnej podstruktury

Optymalne rozwiązanie problemu jest funkcją optymalnych rozwiązań podproblemów (optymalne rozwiązanie problemu można skonstruować z optymalnych rozwiązań podproblemów).

Żeby metoda dziel i zwyciężaj działała dobrze, muszą być spełnione dwa warunki (przez problem i algorytm):

1. własności optymalnej podstawy,
2. podproblemy są rozłączne.

2 Znajdowanie pary najbliższych punktów

Q – zbiór $n \geq 2$ punktów na płaszczyźnie.

$$p_1 = (x_1, y_1), \quad p_2 = (x_2, y_2).$$

$$d(p_1, p_2) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2} \quad \text{– odległość euklidesowa}$$

Problem: Znaleźć parę punktów z Q w najmniejszej odległości.

Algorytm siłowy (brute force), który sprawdza wszystkie pary punktów, działa w czasie $O(n^2)$, bo mamy $\binom{n}{2} = \frac{n(n-1)}{2} = O(n^2)$ par punktów.

Podamy algorytm typu “dziel i zwyciężaj”, który działa w czasie $O(n \log n)$.

2.1 Opis algorytmu

W każdym wywołaniu rekurencyjnym wejście składa się ze zbioru $P \subseteq Q$ i dwóch tablic X, Y , które zawierają wszystkie punkty z P .

W tablicy X punkty są posortowane w porządku niemalejącym względem pierwszej współrzędnej, a w tablicy Y punkty są posortowane w porządku niemalejącym względem drugiej współrzędnej.

Jeśli $|P| \leq 3$, to sprawdzamy wszystkie pary.

Dla $|P| > 3$:

Dziel: Dzielimy punkty z P pionową prostą l na dwa podzbiory P_L i P_R , na lewo i prawo od l , tak że $P_L = \lceil \frac{|P|}{2} \rceil$ i $P_R = \lfloor \frac{|P|}{2} \rfloor$.

Dzielimy wtedy X na X_L i X_R zawierające punkty odpowiednio z P_L i P_R , posortowane w porządku niemalejącym pierwszej współrzędnej. Podobnie dzielimy Y na Y_L i Y_R zawierające punkty z P_L i P_R , posortowane w porządku niemalejącym drugiej współrzędnej.

Zwyciężaj: Wywołujemy rekurencyjnie algorytm dla P_L, X_L, Y_L i P_R, X_R, Y_R . Algorytm znajduje pary najbliższych punktów w P_L i P_R . Niech δ_L i δ_R będą najmniejszymi odległościami znalezionymi przez algorytm dla P_L i P_R odpowiednio.

Niech $\delta = \min(\delta_L, \delta_R)$.

Połącz: Para najbliższych punktów w P składa się z dwóch punktów z P_L , lub dwóch punktów z P_R , lub jednego z P_L i jednego z P_R .

Musimy sprawdzić, czy istnieje para trzeciego typu w odległości mniejszej niż δ . Jeśli taka para istnieje, wtedy oba jej punkty są w pionowym pasie o szerokości 2δ wokół l .

W celu sprawdzenia, czy taka para istnieje, wykonujemy następujące czynności:

1. Tworzymy tablicę Y' otrzymaną z Y poprzez usunięcie punktów spoza pasa. Y' jest posortowany w porządku niemalejącym względem drugiej współrzędnej.
2. Dla każdego punktu p z Y' sprawdzamy odległość między p a p' dla 7 kolejnych punktów p' z Y' .

Algorytm oblicza najmniejszą odległość δ' , zapamiętuje ją i odpowiednią parę punktów.

3. Jeśli $\delta' < \delta$, to pionowy pas zawiera parę punktów o mniejszej odległości niż pary znalezione w wywołaniach rekurencyjnych. Algorytm zwraca δ' i odpowiednią parę punktów. W przeciwnym przypadku algorytm zwraca δ i odpowiednią parę punktów.

2.2 Analiza poprawności

Jedyna rzecz, jaka nie jest oczywista, jest fakt, że sprawdzamy tylko 7 następnych punktów po p w Y' w fazie Połącz.

Dowód. Przyjmij, że p i q są parą punktów w najmniejszej odległości w P i $d(p, q) = \delta' < \delta$. Niech $p = (x_p, y_p)$, $q = (x_q, y_q)$ i $y_p \leq y_q$.

Pokażemy, że algorytm znajduje tę parę punktów.

Przypuśćmy, że to nieprawda. Wtedy istnieje co najmniej 7 punktów pomiędzy p i q w Y' . Niech $r = (x_r, y_r)$ będzie jednym z nich. Skoro $d(p, q) < \delta$, to

$$y_q - y_p = |y_q - y_p| \leq \sqrt{(x_p - x_q)^2 + (y_p - y_q)^2} = d(p, q) < \delta$$

Ponadto $y_p \leq y_r \leq y_q$, więc $y_r - y_p \leq y_q - y_p < \delta$. Zatem jest to co najmniej 9 punktów z P w prostokącie $2\delta \times \delta$ ograniczonym poziomymi liniami $y = y_p$ i $y = y_p + \delta$.

Wtedy w lewym kwadracie $\delta \times \delta$ lub w prawym kwadracie $\delta \times \delta$ jest przynajmniej 5 punktów z P . Przypuśćmy, że to lewy kwadrat.

Zatem istnieje kwadrat $\delta \times \delta$ z 5 punktami z P w P_L . Ale wtedy istnieją 2 punkty z P_L w odległości $< \delta$ w P_L , bo nie da się umieścić w kwadracie $\delta \times \delta$ 5 punktów, dla których odległość między dwoma dowolnymi jest $\geq \delta$ – sprzeczność (bo najmniejsze odległości w P_L to $\delta_L \geq \delta$).

Zatem wystarczy sprawdzić tylko 7 następnych punktów po p w Y' . □

2.3 Analiza złożoności czasowej

Na początku musimy posortować tablice X i Y – to zajmuje $O(n \log n)$.

Jeśli X i Y są posortowane, to podział X na posortowane X_L i X_R oraz podział Y na posortowane Y_L i Y_R kosztuje $O(n)$.

Mamy dwa wywołania rekurencyjne dla $\frac{n}{2}$ punktów i fazę Połącz. Tablicę Y' można otrzymać z Y w czasie $O(n)$, tak że Y' jest posortowana. Niech x_l będzie równaniem prostej l .

Skoro dla każdego z $O(n)$ punktów w Y' sprawdzamy odległość tylko do co najwyżej 7 punktów, to najmniejszą odległość δ' punktów z Y' znajdujemy w czasie $O(n)$.

$T(n)$ – złożoność czasowa algorytmu bez wstępnego sortowania:

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + O(n)$$

$$a = 2, b = 2, k = 1, \quad 2 = 2^1 \implies T(n) = \Theta\left(n^{\log_2 2} \log n\right) = \Theta(n \log n)$$

Złożoność całego algorytmu wynosi $\Theta(n \log n) + \Theta(n \log n) = O(n \log n)$.

Notatki z wykładu: Algorytmy zachłanne

Problem wyboru zajęć (Activity Selection Problem)

1 kwietnia 2026

1 Wstęp do algorytmów zachłannych

Pomysł: Algorytm w każdym kroku dokonuje **lokalnie najlepszego wyboru** (wybiera element, który w danym momencie wydaje się najkorzystniejszy) [1, 7]. Celem jest, aby seria takich wyborów doprowadziła do rozwiązania optymalnego globalnie [7, 8]. Choć strategia ta nie zawsze działa, jest bardzo efektywna dla określonych problemów o strukturze zachłannej [8, 9].

2 Problem wyboru zajęć

2.1 Definicja problemu

Mamy zbiór n zajęć $S = \{z_1, z_2, \dots, z_n\}$ wymagających dostępu do tego samego, niepodzielnego zasobu (np. sala lekcyjna, procesor) [1, 8]. Każde zajęcie z_i charakteryzują [1, 9]:

- s_i – czas rozpoczęcia zajęcia,
- t_i – czas zakończenia zajęcia ($s_i < t_i$).

Zajęcia z_i oraz z_j nazywamy **zgodnymi**, jeśli ich przedziały czasowe są rozłączne: $(s_i, t_i) \cap (s_j, t_j) = \emptyset$ [1, 9].

Cel: Znaleźć podzbiór $A \subseteq S$ o **największej liczności** (maksymalnej liczbie zajęć), składający się z zajęć parami zgodnych [1, 9].

2.2 Przykład z wykładu

Dane dla $n = 6$ zajęć przedstawione na tablicy [1, 5, 10]:

i	1	2	3	4	5	6
s_i	1	0	2	4	7	7
t_i	3	4	6	7	9	10

Wstępną fazą algorytmu jest **sortowanie zajęć** w porządku niemalejącym według czasu zakończenia: $t_1 \leq t_2 \leq \dots \leq t_n$. Zajmuje to czas $O(n \log n)$ [1, 3, 10].

3 Podejście 1: Programowanie dynamiczne

3.1 Definicja podproblemu

Definiujemy S_{ij} jako zbiór zajęć zaczynających się po zakończeniu z_i i kończących przed rozpoczęciem z_j [2, 11]:

$$S_{ij} = \{z_k \in S : t_i \leq s_k < t_k \leq s_j\}$$

Dodajemy dwa sztuczne zajęcia: z_0 ($t_0 = 0$) oraz z_{n+1} ($s_{n+1} = \infty$). Wtedy cały problem $S = S_{0,n+1}$ [2, 11].

3.2 Własność optymalnej podstruktury i rekurencja

Jeśli z_k należy do rozwiązania optymalnego A_{ij} , to $A_{ij} = A_{ik} \cup \{z_k\} \cup A_{kj}$, gdzie A_{ik} i A_{kj} są rozwiązaniami optymalnymi dla mniejszych podproblemów [2, 12]. Niech $c[i, j] = |A_{ij}|$. Wzór rekurencyjny to [3, 13]:

$$c[i, j] = \begin{cases} 0 & \text{dla } i \geq j \text{ lub } S_{ij} = \emptyset \\ \max_{i < k < j, z_k \in S_{ij}} \{c[i, k] + c[k, j] + 1\} & \text{w przeciwnym razie} \end{cases}$$

Liczba podproblemów wynosi $\Theta(n^2)$, a koszt jednego to $O(n)$, co daje całkowitą złożoność $\Theta(n^3)$ [3, 13, 14].

4 Podejście 2: Algorytm zachłanny

Algorytm zachłanny zawsze wybiera zajęcie kończące się najwcześniej, co pozostawia najwięcej miejsca dla przyszłych zajęć [4, 15, 16].

Lemat 1 (O wyborze zachłannym). *Niech $z_m \in S_{ij}$ będzie zajęciem o najwcześniejszym czasie zakończenia ($t_m = \min\{t_k : z_k \in S_{ij}\}$). Wtedy [4, 15]:*

1. z_m należy do pewnego optymalnego rozwiązania dla S_{ij} .
2. Podproblem $S_{im} = \emptyset$. Wybór z_m redukuje problem do jednego podproblemu S_{mj} .

4.1 Pseudokod algorytmu

Algorithm 1 Activity-Selection(s, t)

```
 $n \leftarrow \text{length}(s)$ 
 $A \leftarrow \{z_1\}$  {Wybieramy zajęcie o najwcześniejszym końcu}
 $j \leftarrow 1$  {Indeks ostatnio dodanego zajęcia}
for  $i \leftarrow 2$  to  $n$  do
  if  $s_i \geq t_j$  then
     $A \leftarrow A \cup \{z_i\}$ 
     $j \leftarrow i$ 
  end if
end for
return  $A$ 
```

Dla przykładu z tablicy algorytm wybierze $A = \{z_1, z_4, z_5\}$ [5, 17].

5 Analiza poprawności (Dowód indukcyjny)

Pokażemy, że zbiór A wyznaczony przez algorytm jest optymalny [6, 18].

- **Baza:** Dla $n = 1$ algorytm wybiera z_1 , co jest oczywiście optymalne [6, 18].
- **Krok indukcyjny ($n > 1$):** Z Lematu wiemy, że istnieje optymalne rozwiązanie A_1 zawierające z_1 . Rozważmy podproblem $S' = \{z_k \in S : s_k \geq t_1\}$ [6, 19].
- Zbiór $A_1 \setminus \{z_1\}$ musi być rozwiązaniem optymalnym dla S' . Gdyby istniało lepsze rozwiązanie B dla S' ($|B| > |A_1 \setminus \{z_1\}|$), to zbiór $\{z_1\} \cup B$ byłby lepszy dla S niż A_1 , co przeczy optymalności A_1 [6, 19].
- Z założenia indukcyjnego nasz algorytm znajduje rozwiązanie optymalne dla mniejszego problemu S' . Łącząc z_1 z optymalnym rozwiązaniem dla S' , otrzymujemy rozwiązanie optymalne dla całego zbioru S [6, 20, 21].

6 Złożoność czasowa

- **Inicjalizacja i pętla:** $O(n)$ – przechodzimy przez zajęcia tylko raz [6, 22].
- **Sortowanie:** $O(n \log n)$ – faza wstępna [3, 22].
- **Całkowita złożoność:** $O(n \log n)$. Jest to znaczna poprawa względem $\Theta(n^3)$ z programowania dynamicznego [6, 23].

Notatki z wykładu: Algorytmy zachłanne

Kody Huffmana

8 kwietnia 2026

1 Motywacja: kodowanie znaków

Często znaki są reprezentowane przy użyciu **8 bitów** (np. kod ASCII), czyli przy pomocy słów kodowych o *stałej* długości. Można jednak kodować oszczędniej, jeśli zezwolimy na słowa kodowe o *zmiennej* długości.

Pomysł: krótkie sekwencje bitów lepiej reprezentują znaki *występujące często*, natomiast znaki *występujące rzadko* reprezentujemy przy użyciu dłuższych sekwencji. W ten sposób minimalizujemy łączną liczbę bitów potrzebnych do zakodowania pliku.

2 Przykład wprowadzający

Rozważmy plik składający się z 58 znaków, w którym każdy znak jest elementem zbioru

$$\{a, e, i, s, t, r, n\}.$$

Częstości wystąpień poszczególnych znaków oraz trzy różne kody zebrano w tabeli:

		<i>a</i>	<i>e</i>	<i>i</i>	<i>s</i>	<i>t</i>	<i>r</i>	<i>n</i>
częstość (liczba wystąpień)		10	15	12	3	4	13	1
kod 1	stała długość	000	001	010	011	100	101	110
kod 2	zmienna długość	001	01	10	00000	0001	11	00001
kod 3	zmienna długość	001	01	10	00010	0001	11	00001

Suma częstości wynosi $10 + 15 + 12 + 3 + 4 + 13 + 1 = 58$, zgodnie z długością pliku.

Kod o stałej długości (kod 1)

Każdy znak zajmuje 3 bity, więc zakodowany plik ma rozmiar

$$3 \cdot 58 = 174 \text{ bity.}$$

Kod o zmiennej długości (kod 2)

Rozmiar pliku to suma iloczynów częstości znaku i długości jego słowa kodowego:

$$10 \cdot 3 + 15 \cdot 2 + 12 \cdot 2 + 3 \cdot 5 + 4 \cdot 4 + 13 \cdot 2 + 1 \cdot 5 = 146 \text{ bitów.}$$

Kod o zmiennej długości pozwala więc zaoszczędzić $174 - 146 = 28$ bitów względem kodu o stałej długości.

3 Kody prefiksowe

Jeśli używamy kodu o zmiennej długości słowa kodowego, to chcemy, aby spełniony był warunek gwarantujący *jednoznaczność dekodowania*.

Definicja 1 (kod prefiksowy). Kod nazywamy **prefiksowym**, jeśli żadne słowo kodowe nie jest prefiksem innego słowa kodowego.¹

Tak długo jak warunek prefiksowości jest spełniony, ciąg bitów daje się odkodować jednoznacznie.

Dlaczego kod 3 jest zły

Kod 3 *nie* jest prefiksowy: słowo kodowe znaku t równe 0001 jest prefiksem słowa kodowego znaku s równego 00010. Rozważmy ciąg bitów

0001001.

Ciąg ten można odkodować na (co najmniej) dwa sposoby:

$$\underbrace{00010}_s \underbrace{01}_e \quad \text{albo} \quad \underbrace{0001}_t \underbrace{001}_a.$$

Nie możemy zatem jednoznacznie odkodować pliku — właśnie dlatego, że 0001 jest prefiksem 00010.

4 Reprezentacja drzewowa kodów

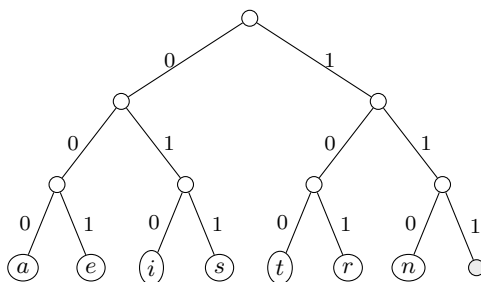
Każdy kod możemy przedstawić jako **drzewo binarne**: schodząc w lewo dopisujemy bit 0, a w prawo bit 1. Symbol jest wówczas ciągiem bitów odczytanym na ścieżce od korzenia (ścieżce wyszukiwania).

Uwaga. Kod jest prefiksowy $\iff$ każdy symbol w reprezentacji drzewowej znajduje się w *liściu*.

Wierzchołki etykietujemy w następujący sposób: każdy wierzchołek v dostaje etykietę równą sumie częstości wystąpień znaków reprezentowanych przez liście poddrzewa o korzeniu v .

Kod 1 — o stałej długości słowa kodowego

Wszystkie liście leżą na tej samej głębokości 3:

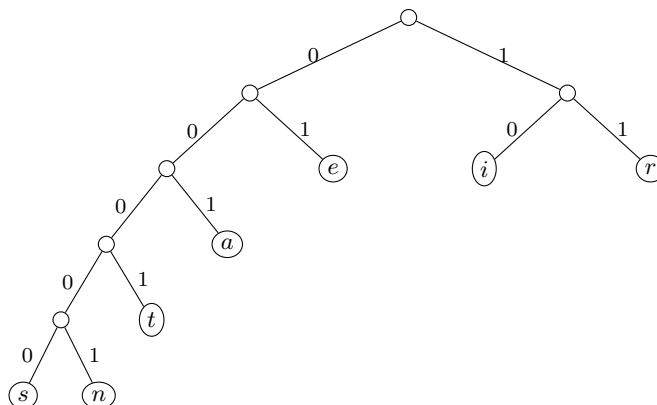


(szary wierzchołek odpowiada nieużywanemu słowu 111).

¹Poprawniejszą (choć rzadziej używaną) nazwą byłoby *kod bezprefiksowy* — nazwa „prefiksowy” utrwalila się umownie.

Kod 2 — o zmiennej długości słowa kodowego

Tu znaki o większej częstości (e, i, r) leżą płycej, a znaki rzadkie (s, n, t) — głębiej:



5 Koszt kodu i optymalność

Niech $\mathbb{C}$ będzie **alfabetem** (zbiorem znaków), a f funkcją częstości

$$f : \mathbb{C} \rightarrow \mathbb{N}, \quad f(c) = \text{częstość znaku } c \in \mathbb{C}.$$

Dla drzewa T reprezentującego kod oznaczmy przez

$$d_T(c) = \text{głębokość liścia reprezentującego znak } c \text{ w drzewie } T$$

długość słowa kodowego znaku c w kodzie reprezentowanym przez drzewo T . Wtedy liczba bitów w zakodowanym pliku wynosi

$$b_T = \sum_{c \in \mathbb{C}} d_T(c) f(c).$$

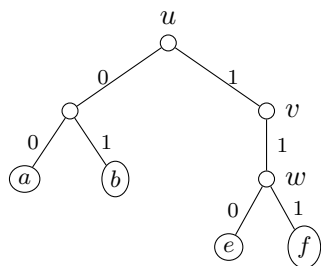
Definicja 2 (kod optymalny). Kod prefiksowy reprezentowany przez drzewo T nazywamy **optymalnym**, jeśli liczba b_T jest najmniejsza możliwa.

Definicja 3 (drzewo binarne i regularne). Drzewo ukorzenione nazywamy **binarnym**, jeśli każdy wierzchołek ma co najwyżej dwoje dzieci. Drzewo binarne nazywamy **regularnym**, jeśli każdy wierzchołek wewnętrzny ma dokładnie dwoje dzieci.

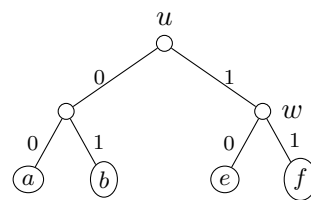
6 Drzewo optymalnego kodu jest regularne

Twierdzenie 1. *Drzewo binarne reprezentujące optymalny kod jest regularne.*

Dowód. Niech T będzie drzewem binarnym reprezentującym optymalny kod. Załóżmy, że *nie* jest ono regularne. Wtedy istnieje wierzchołek wewnętrzny v , który ma dokładnie jedno dziecko w . Jeśli v nie jest korzeniem, to niech u będzie rodzicem wierzchołka v .



T



T'

Niech T' będzie drzewem powstałym z T przez usunięcie v i połączenie u z w (a jeśli v był korzeniem — przez przyjęcie w za nowy korzeń). W T' głębokości wszystkich liści poddrzewa zakorzenionego w w zmalały o 1, a pozostałe liście nie zmieniły głębokości. Zatem

$$b(T') < b(T),$$

co przeczy optymalności T . Otrzymana sprzeczność dowodzi tezy. $\square$

7 Algorytm Huffmana

Algorytm Huffmana konstruuje drzewo reprezentujące optymalny kod prefiksowy dla danego wejścia: alfabetu $\mathbb{C}$ oraz funkcji częstości f .

Idea. Zaczynamy od $|\mathbb{C}|$ wierzchołków izolowanych, które staną się liśćmi, i wykonujemy $|\mathbb{C}| - 1$ operacji *scalania*, zaczynając od najrzadszych znaków. Znaki o najmniejszej częstości trafiają wówczas najgłębiej w drzewie.

Struktura danych. Będziemy używać **kolejki priorytetowej** q z kluczem $f(c)$. Kolejkę priorytetową możemy zaimplementować przy użyciu *kopca binarnego*. Wtedy koszty operacji są następujące:

$$\underbrace{\text{INSERT}}_{O(\log n)}, \quad \underbrace{\text{EXTRACT-MIN}}_{O(\log n)}, \quad \underbrace{\text{CREATE}}_{O(n)}$$

gdzie $n = |\mathbb{C}|$.

Oznaczenia w pseudokodzie: z — nowo tworzony wierzchołek, $left(z)$ i $right(z)$ — jego dzieci, q — kolejka priorytetowa.

Algorytm konstruuje drzewo binarne za pomocą $n-1$ operacji scalania, w którym każdy wierzchołek wewnętrzny ma dokładnie dwoje dzieci (drzewo jest regularne).

8 Przykład działania algorytmu

Wracamy do alfabetu $\{a, e, i, s, t, r, n\}$ z częstościami z tabeli. Algorytm w kolejnych krokach scala dwa elementy o najmniejszych kluczach:

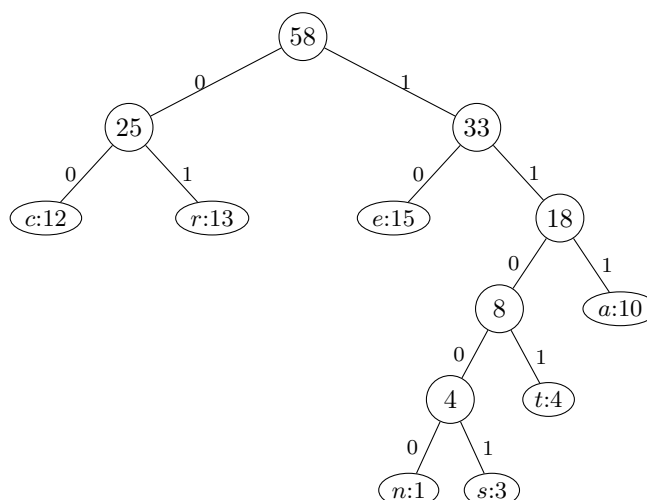
Algorithm 1 Huffman($\mathbb{C}, f$)

```
1:  $n \leftarrow |\mathbb{C}|$  {liczba znaków;  $O(n)$ }
2: utwórz kolejkę  $q$  na podstawie  $f$  { $O(n)$ }
3: for  $i \leftarrow 1$  to  $n - 1$  do
4:   utwórz nowy wierzchołek  $z$  {pętla wykonuje się  $n - 1$  razy}
5:    $x \leftarrow \text{left}(z) \leftarrow \text{EXTRACT-MIN}(q)$  { $O(\log n)$ }
6:    $y \leftarrow \text{right}(z) \leftarrow \text{EXTRACT-MIN}(q)$  { $O(\log n)$ }
7:    $f(z) \leftarrow f(x) + f(y)$ 
8:    $\text{INSERT}(q, z)$  { $O(\log n)$ }
9: end for
10: return  $\text{EXTRACT-MIN}(q)$  {korzeń drzewa}
```

krok	scalane elementy	nowy wierzchołek
0	— (start)	$n:1, s:3, t:4, a:10, c:12, r:13, e:15$
1	$n:1$ oraz $s:3$	4
2	4 oraz $t:4$	8
3	8 oraz $a:10$	18
4	$c:12$ oraz $r:13$	25
5	18 oraz $e:15$	33
6	25 oraz 33	58 (korzeń)

Uwaga. W tym przykładzie częstość $c:12$ przyjęto dodatkowo (rozszerzony zestaw znaków z wykładu); kluczowe jest, że w każdym kroku scalamy dwa najmniejsze klucze.

Drzewo końcowe



Wynikowe słowa kodowe

Odczytując ścieżki od korzenia (0 w lewo, 1 w prawo) otrzymujemy:

znak	słowo kodowe	długość
<i>c</i>	00	2
<i>r</i>	01	2
<i>e</i>	10	2
<i>a</i>	111	3
<i>t</i>	1101	4
<i>n</i>	11000	5
<i>s</i>	11001	5

Łączna liczba bitów dla tego kodu:

$$12 \cdot 2 + 13 \cdot 2 + 15 \cdot 2 + 10 \cdot 3 + 4 \cdot 4 + 1 \cdot 5 + 3 \cdot 5 = 146 \text{ bitów},$$

czyli tyle samo, co dla kodu 2 — oba kody są optymalne.

9 Złożoność czasowa

Pętla wykonuje się $n - 1$ razy, a jej wnętrze (dwie operacje EXTRACT-MIN oraz jedna INSERT) kosztuje $O(\log n)$. Stąd

$$(n - 1) \cdot O(\log n) = O(n \log n).$$

Wraz z kosztem inicjalizacji $O(n)$ całkowita złożoność algorytmu Huffmana wynosi

$$\boxed{O(n \log n)}.$$

Szeregowanie zadań. Dowody optymalności kodów Huffmana – Wykład

1 Prostý problem szeregowania zadań

$\{z_1, z_2, \dots, z_n\}$ – zbiór zadań.

Każde zadanie z_i ma swój czas wykonania t_i , $i \in [n]$.

Rozpoczęte zadanie nie może być przerwane.

1.1 Problem 1

Mamy jeden procesor.

Jak uszeregować zadania tak, aby średni czas ukończenia był jak najmniejszy?

Przykład 1.

i	zadanie	t_i
1	z_1	15
2	z_2	8
3	z_3	3
4	z_4	10

Kolejność z_1, z_2, z_3, z_4 :

$$\frac{15 + (15 + 8) + (15 + 8 + 3) + (15 + 8 + 3 + 10)}{4} = \frac{15 + 23 + 26 + 36}{4} = \frac{100}{4} = 25$$

Kolejność z_3, z_2, z_4, z_1 (od najkrótszego do najdłuższego):

$$\frac{3 + (3 + 8) + (3 + 8 + 10) + (3 + 8 + 10 + 15)}{4} = \frac{3 + 11 + 21 + 36}{4} = \frac{71}{4} = 17,75$$

Rozwiązanie: Przydzielaj zadania do procesora w kolejności od najkrótszego do najdłuższego.

1.2 Problem 2

Mamy 3 procesory.

Pytanie: jak poprawnie?

Przykład 2.

zadanie	z_1	z_2	z_3	z_4	z_5	z_6	z_7	z_8
t_i	3	5	6	10	11	15	18	20

Rozwiązanie: Przydzielaj zadania do pierwszego wolnego procesora w kolejności od najkrótszego do najdłuższego.

Przydział na 3 procesory:

- P_1 : z_1 (3), z_4 (10), z_7 (18)
- P_2 : z_2 (5), z_5 (11), z_8 (20)
- P_3 : z_3 (6), z_6 (15)

1.3 Dowód poprawności

Zatem $f(\Pi') \leq f(\Pi)$, ale Π minimalizuje f , więc $f(\Pi') = f(\Pi)$ i Π' też minimalizuje f .

Po pewnej liczbie takich zamian otrzymujemy permutację $\Pi_0 = (j_1, \dots, j_n)$ minimalizującą f taką, że

$$t_{j_1} \leq t_{j_2} \leq \dots \leq t_{j_n}$$

Wniosek 1. Nasz algorytm szeregowania zadań jest poprawny.

2 Dowody optymalności kodów Huffmana

Kody prefiksowe $\longleftrightarrow$ regularne drzewa binarne.

C – alfabet, $f: C \rightarrow \mathbb{N}$ – funkcja częstości, $d_T(c)$ – głębokość liścia reprezentującego znak c w drzewie T reprezentującym kod prefiksowy.

$$B(T) = \sum_{c \in C} f(c) \cdot d_T(c) \longleftarrow \min$$

Lemat 1. Niech x i y będą najrzadziej występującymi znakami z C . Istnieje optymalny kod prefiksowy, w którym słowa kodowe x i y mają tę samą długość i różnią się tylko na ostatnim bicie.

Dowód. Znajdziemy drzewo reprezentujące optymalny kod prefiksowy, w którym liście reprezentujące x i y mają wspólnego rodzica i mają największą głębokość w drzewie.

Niech T będzie dowolnym drzewem reprezentującym optymalny kod prefiksowy i niech a i b będą dwoma liśćmi o tym samym rodzicu i największej głębokości w T .

Bez straty ogólności możemy przyjąć, że $f(x) \leq f(y)$ i $f(a) \leq f(b)$.

$f(x)$ i $f(y)$ są najmniejszymi częstościami, więc $f(x) \leq f(a)$ i $f(y) \leq f(b)$.

Niech T' będzie drzewem powstałym z T przez zamianę liści x i a .

$$\begin{aligned} B(T) - B(T') &= \sum_{c \in C} f(c) d_T(c) - \sum_{c \in C} f(c) d_{T'}(c) \\ &= f(a) d_T(a) + f(x) d_T(x) - f(a) d_{T'}(a) - f(x) d_{T'}(x) \\ &= f(a) d_T(a) + f(x) d_T(x) - f(a) d_T(x) - f(x) d_T(a) \\ &= f(a)(d_T(a) - d_T(x)) - f(x)(d_T(a) - d_T(x)) \\ &= (f(a) - f(x))(d_T(a) - d_T(x)) \geq 0 \end{aligned}$$

(bo $f(a) \geq f(x)$ i $d_T(a) \geq d_T(x)$, gdyż a ma największą głębokość).

Zatem $B(T') \leq B(T)$, ale $B(T)$ jest najmniejsze możliwe, więc $B(T') = B(T)$ – czyli T' też reprezentuje optymalny kod prefiksowy.

Analogicznie zamieniamy miejscami liście y i b i otrzymujemy drzewo T'' reprezentujące optymalny kod prefiksowy, które ma żądane własności. $\square$

Z Lematu 1 wynika, że można zacząć konstruując optymalne drzewo połączeniem dwóch symboli o najmniejszej częstości.

Zachłanny wybór – w każdym kroku algorytm łączy dwa wierzchołki o najmniejszych kluczach.

Lemat 2. Niech A będzie alfabetem, a $f: A \rightarrow \mathbb{N}$ funkcją częstości. Niech x i y będą dwoma najrzadziej występującymi znakami w A . Niech $A' = (A \setminus \{x, y\}) \cup \{z\}$, gdzie $z \notin A$, i $f': A' \rightarrow \mathbb{N}$ będzie funkcją częstości taką, że:

- dla każdego $c \in A' \setminus \{z\}$: $f'(c) = f(c)$,

- $f'(z) = f(x) + f(y)$.

$|C'| = n - 1$. Z założenia indukcyjnego algorytm znajduje drzewo T' reprezentujące optymalny kod prefiksowy dla (A', f') .

Niech T będzie drzewem powstałym z T' poprzez zastąpienie liścia z wierzchołkiem wewnętrznym z dziećmi x i y .

T reprezentuje optymalny kod prefiksowy dla (A, f) .

Dowód. Dla $c \in A \setminus \{x, y\}$: $d_T(c) = d_{T'}(c)$, a $d_T(x) = d_T(y) = d_{T'}(z) + 1$.

Zatem:

$$\begin{aligned}
B(T) &= \sum_{c \in A} f(c) \cdot d_T(c) \\
&= \sum_{c \in A \setminus \{x, y\}} f(c) d_T(c) + f(x) d_T(x) + f(y) d_T(y) \\
&= \sum_{c \in A' \setminus \{z\}} f'(c) d_{T'}(c) + f(x)(d_{T'}(z) + 1) + f(y)(d_{T'}(z) + 1) \\
&= \sum_{c \in A' \setminus \{z\}} f'(c) d_{T'}(c) + d_{T'}(z)(f(x) + f(y)) + f(x) + f(y) \\
&= \sum_{c \in A'} f'(c) d_{T'}(c) + f(x) + f(y) \\
&= B(T') + f(x) + f(y)
\end{aligned}$$

Stąd:

$$B(T') = B(T) - f(x) - f(y)$$

Przypuśćmy, że T nie reprezentuje optymalnego kodu dla (A, f) . Niech T'' będzie drzewem reprezentującym optymalny kod prefiksowy dla (A, f) , tzn. $B(T'') < B(T)$.

Z Lematu 1 możemy wybrać T'' tak, aby x i y miały wspólnego rodzica i największą głębokość w T'' . Niech T''' będzie drzewem powstałym z T'' poprzez usunięcie liści x i y , nadanie ich rodzicowi etykiety z z kluczem $f(x) + f(y)$.

T''' reprezentuje kod prefiksowy dla (A', f') .

Takim samym obliczeniem jak wcześniej (zamieniając wszędzie T na T'' , T' na T''') pokazujemy, że $B(T''') = B(T'') - f(x) - f(y)$.

Mamy:

$$B(T''') = B(T'') - f(x) - f(y) < B(T) - f(x) - f(y) = B(T')$$

Sprzeczność z optymalnością T' dla (A', f') .

Zatem T reprezentuje optymalny kod dla (A, f) . □

Twierdzenie 1. Algorytm Huffmana konstruuje drzewo reprezentujące optymalny kod prefiksowy.

Dowód. Indukcja po n = liczba symboli w alfabecie C .

Dla $n = 1, 2$ – oczywiste.

Niech $n > 2$. Po pierwszej operacji łączenia dostajemy alfabet $C' = (C \setminus \{x, y\}) \cup \{z\}$ i funkcję $f': C' \rightarrow \mathbb{N}$ taką, że $f'(c) = f(c)$ dla wszystkich $c \in C' \setminus \{z\}$ i $f'(z) = f(x) + f(y)$, gdzie x i y są znakami z C o najmniejszych częstościach.

$|C'| = n - 1$. Z założenia indukcyjnego algorytm znajduje drzewo T' reprezentujące optymalny kod prefiksowy dla (C', f') .

Niech T będzie drzewem powstałym z T' poprzez zastąpienie liścia z wierzchołkiem wewnętrznym z dziećmi x i y . Z Lematu 2: T reprezentuje optymalny kod prefiksowy dla (A, f) . □

Kody Huffmana (dokończenie) i szeregowanie zadań

1 Optymalne kody prefiksowe – przypomnienie

Przypomnijmy, że kody prefiksowe odpowiadają wzajemnie jednoznacznie *regularnym drzewom binarnym* (każdy wierzchołek wewnętrzny ma dokładnie dwoje dzieci), w których znaki alfabetu odpowiadają liściom.

Niech C będzie alfabetem, a

$$f : C \rightarrow \mathbb{N}$$

funkcją częstości (liczba wystąpień znaku). Dla drzewa T reprezentującego kod prefiksowy oznaczmy przez $d_T(c)$ głębokość liścia reprezentującego znak c w drzewie T . Głębokość liścia jest jednocześnie długością słowa kodowego znaku c .

Koszt drzewa T nazywamy liczbę bitów potrzebnych do zakodowania pliku:

$$B(T) = \sum_{c \in C} f(c) \cdot d_T(c) \longleftarrow \min.$$

Naszym celem jest znalezienie drzewa T minimalizującego $B(T)$ – takie drzewo nazywamy **optymalnym kodem prefiksowym**.

2 Własności optymalnego kodu prefiksowego

Lemat 1. Niech x i y będą najrzadziej występującymi znakami z C . Wtedy istnieje optymalny kod prefiksowy, w którym słowa kodowe x i y mają tę samą długość i różnią się tylko na ostatnim bicie.

Innymi słowy: istnieje optymalne drzewo, w którym liście reprezentujące x i y mają wspólnego rodzica i znajdują się na największej głębokości w drzewie.

Dowód. Znajdziemy drzewo reprezentujące optymalny kod prefiksowy, w którym liście reprezentujące x i y mają wspólnego rodzica i mają największą głębokość w drzewie.

Niech T będzie dowolnym drzewem reprezentującym optymalny kod prefiksowy i niech a oraz b będą dwoma liśćmi o tym samym rodzicu i największej głębokości w T (takie dwa liście istnieją, bo drzewo jest regularne, więc najgłębszy liść ma rodzeństwo).

Bez straty ogólności możemy przyjąć, że

$$f(x) \leq f(y) \quad \text{oraz} \quad f(a) \leq f(b).$$

Ponieważ $f(x)$ i $f(y)$ są najmniejszymi częstościami, to

$$f(x) \leq f(a) \quad \text{oraz} \quad f(y) \leq f(b).$$

Niech T' będzie drzewem powstałym z T przez zamianę miejscami liści x i a . Wówczas $d_{T'}(a) = d_T(x)$ oraz $d_{T'}(x) = d_T(a)$, a położenia pozostałych liści się nie zmieniają. Liczymy

różnicę kosztów:

$$\begin{aligned}
B(T) - B(T') &= \sum_{c \in C} f(c) d_T(c) - \sum_{c \in C} f(c) d_{T'}(c) \\
&= f(a) d_T(a) + f(x) d_T(x) - f(a) d_{T'}(a) - f(x) d_{T'}(x) \\
&= f(a) d_T(a) + f(x) d_T(x) - f(a) d_T(x) - f(x) d_T(a) \\
&= f(a)(d_T(a) - d_T(x)) - f(x)(d_T(a) - d_T(x)) \\
&= \underbrace{(f(a) - f(x))}_{\geq 0} \underbrace{(d_T(a) - d_T(x))}_{\geq 0} \geq 0.
\end{aligned}$$

Pierwszy czynnik jest nieujemny, bo $f(x) \leq f(a)$. Drugi czynnik jest nieujemny, bo a jest liściem o największej głębokości, więc $d_T(a) \geq d_T(x)$.

Zatem $B(T') \leq B(T)$, ale $B(T)$ jest najmniejszą możliwą wartością (bo T jest optymalne), więc $B(T') = B(T)$, czyli T' też reprezentuje optymalny kod prefiksowy.

Analogicznie zamieniamy miejscami liście y i b i otrzymujemy drzewo T'' reprezentujące optymalny kod prefiksowy, które ma żądane własności: liście x i y są rodzeństwem na największej głębokości. $\square$

Z Lematu 1 wynika, że konstrukcję optymalnego drzewa można rozpocząć od połączenia dwóch symboli o najmniejszej częstości – ich liście na pewno będą rodzeństwem w pewnym drzewie optymalnym.

Zachłanny wybór. W każdym kroku algorytm łączy dwa wierzchołki o najmniejszych częstościach w jeden nowy wierzchołek (poddzewo), którego częstość jest sumą częstości połączonych wierzchołków.

Lemat 2. Niech A będzie alfabetem, a $f : A \rightarrow \mathbb{N}$ funkcją częstości. Niech x i y będą dwoma najrzadziej występującymi znakami w A . Niech

$$A' = (A \setminus \{x, y\}) \cup \{z\}, \quad z \notin A,$$

a $f' : A' \rightarrow \mathbb{N}$ będzie funkcją częstości taką, że dla każdego $c \in A' \setminus \{z\}$

$$f'(c) = f(c) \quad \text{oraz} \quad f'(z) = f(x) + f(y).$$

Niech T' będzie drzewem reprezentującym optymalny kod prefiksowy dla (A', f') . Niech T będzie drzewem powstałym z T' poprzez zastąpienie liścia z wierzchołkiem wewnętrznym z dziećmi x i y . Wówczas T reprezentuje optymalny kod prefiksowy dla (A, f) .

Dowód. Dla każdego $c \in A \setminus \{x, y\}$ mamy $d_T(c) = d_{T'}(c)$, natomiast

$$d_T(x) = d_T(y) = d_{T'}(z) + 1,$$

bo liście x i y pojawiają się o jeden poziom niżej niż dawny liść z .

Obliczamy koszt drzewa T :

$$\begin{aligned}
B(T) &= \sum_{c \in A} f(c) d_T(c) = \sum_{c \in A \setminus \{x, y\}} f(c) d_T(c) + f(x) d_T(x) + f(y) d_T(y) \\
&= \sum_{c \in A \setminus \{x, y\}} f'(c) d_{T'}(c) + f(x)(d_{T'}(z) + 1) + f(y)(d_{T'}(z) + 1) \\
&= \sum_{c \in A' \setminus \{z\}} f'(c) d_{T'}(c) + d_{T'}(z) (f(x) + f(y)) + (f(x) + f(y)) \\
&= \sum_{c \in A' \setminus \{z\}} f'(c) d_{T'}(c) + d_{T'}(z) f'(z) + (f(x) + f(y)) \\
&= \sum_{c \in A'} f'(c) d_{T'}(c) + f(x) + f(y) \\
&= B(T') + f(x) + f(y).
\end{aligned}$$

Stąd

$$B(T') = B(T) - f(x) - f(y).$$

Pozostaje pokazać, że T jest optymalne dla (A, f) . Przypuśćmy przeciwnie, że istnieje drzewo T_{opt} dla (A, f) takie, że $B(T_{\text{opt}}) < B(T)$. Na mocy Lematu 1 możemy założyć, że x i y są w T_{opt} rodzeństwem. Zastępując ich wspólnego rodzica liściem z otrzymujemy drzewo T'_{opt} dla (A', f') o koszcie

$$B(T'_{\text{opt}}) = B(T_{\text{opt}}) - f(x) - f(y) < B(T) - f(x) - f(y) = B(T'),$$

co przeczy optymalności T' dla (A', f') . Zatem T jest optymalne dla (A, f) . $\square$

3 Algorytm Huffmana

Lematy 1 i 2 uzasadniają poprawność następującego algorytmu zachłannego, który buduje optymalne drzewo „od dołu”, wielokrotnie łącząc dwa wierzchołki o najmniejszej częstości.

Algorithm 1 HUFFMAN(C, f) – budowa optymalnego kodu prefiksowego

Wejście: Alfabet C oraz funkcja częstości $f : C \rightarrow \mathbb{N}$

Wyjście: Korzeń drzewa optymalnego kodu prefiksowego

- 1: $Q \leftarrow$ kolejka priorytetowa zawierająca liście $c \in C$ z kluczem $f(c)$
 - 2: **for** $i = 1$ **to** $|C| - 1$ **do**
 - 3: $x \leftarrow \text{EXTRACT-MIN}(Q)$
 - 4: $y \leftarrow \text{EXTRACT-MIN}(Q)$
 - 5: utwórz nowy wierzchołek z z dziećmi x i y
 - 6: $f(z) \leftarrow f(x) + f(y)$
 - 7: $\text{INSERT}(Q, z)$
 - 8: **end for**
 - 9: **return** $\text{EXTRACT-MIN}(Q)$ {jedyne pozostałe wierzchołki – korzeń}
-

Przy użyciu kopca jako kolejki priorytetowej każda operacja EXTRACT-MIN i INSERT działa w czasie $O(\log |C|)$, więc cały algorytm działa w czasie $O(|C| \log |C|)$.

4 Szeregowanie zadań

Rozważmy teraz **prosty problem szeregowania zadań**. Dany jest zbiór zadań

$$\{z_1, z_2, \dots, z_n\},$$

gdzie każde zadanie z_i ma swój czas wykonania t_i , $i \in [n]$. Rozpoczęte zadanie nie może być przerwane (model bez wywłaszczania).

4.1 Problem 1 – jeden procesor

Problem 1. Mamy jeden procesor. Jak uszeregować zadania tak, aby *średni czas zakończenia* był jak najmniejszy?

Jeżeli zadania wykonywane są w kolejności $j_1, j_2, \dots, j_n$, to czas zakończenia k -tego zadania wynosi $C_k = \sum_{l=1}^k t_{j_l}$, a minimalizujemy

$$f(\Pi) = \frac{1}{n} \sum_{k=1}^n C_k = \frac{1}{n} \sum_{k=1}^n \sum_{l=1}^k t_{j_l}.$$

Przykład 1. Rozważmy cztery zadania:

i	zadanie	t_i
1	z_1	15
2	z_2	8
3	z_3	3
4	z_4	10

Dla kolejności z_1, z_2, z_3, z_4 średni czas zakończenia wynosi

$$\frac{15 + (15 + 8) + (15 + 8 + 3) + (15 + 8 + 3 + 10)}{4} = \frac{15 + 23 + 26 + 36}{4} = \frac{100}{4} = 25.$$

Dla kolejności z_3, z_2, z_4, z_1 (od najkrótszego do najdłuższego) otrzymujemy

$$\frac{3 + (3 + 8) + (3 + 8 + 10) + (3 + 8 + 10 + 15)}{4} = \frac{3 + 11 + 21 + 36}{4} = \frac{71}{4} = 17\frac{3}{4}.$$

Druga kolejność daje istotnie mniejszą wartość.

Rozwiązanie. Uszereguj zadania od najkrótszego do najdłuższego (rosnąco względem t_i).

Twierdzenie 1. Uszeregowanie zadań w kolejności niemalejących czasów wykonania minimalizuje średni czas zakończenia.

Dowód. Skorzystamy z *argumentu wymiany*. Niech $\Pi = (j_1, \dots, j_n)$ będzie permutacją minimalizującą f i przypuśćmy, że nie jest ona posortowana rosnąco. Wtedy istnieją dwa sąsiednie zadania j_k, j_{k+1} takie, że $t_{j_k} > t_{j_{k+1}}$.

Niech Π' powstaje z Π przez zamianę miejscami j_k i j_{k+1} . Wszystkie czasy zakończenia poza C_k pozostają bez zmian, a C_k zmienia się z wartości $(\sum_{l < k} t_{j_l}) + t_{j_k}$ na $(\sum_{l < k} t_{j_l}) + t_{j_{k+1}}$. Zatem

$$f(\Pi') - f(\Pi) = \frac{t_{j_{k+1}} - t_{j_k}}{n} \leq 0,$$

czyli $f(\Pi') \leq f(\Pi)$. Ale Π minimalizuje f , więc $f(\Pi') = f(\Pi)$ i Π' też minimalizuje f .

Po pewnej liczbie takich zamian otrzymujemy permutację $\Pi_0 = (j_1, \dots, j_n)$ minimalizującą f taką, że

$$t_{j_1} \leq t_{j_2} \leq \dots \leq t_{j_n}.$$

Permutacja posortowana rosnąco jest więc optymalna. □

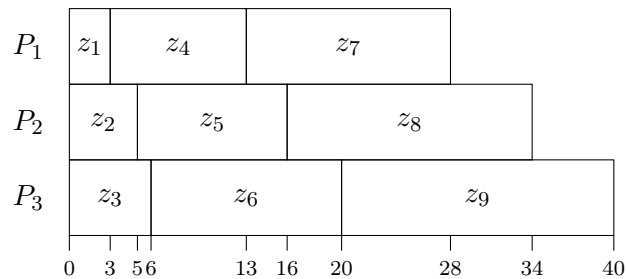
4.2 Problem 2 – wiele procesorów

Problem 2. Mamy 3 procesory. Pytanie jak poprzednio – minimalizujemy średni czas zakończenia.

Przykład 2. Rozważmy dziewięć zadań:

zadanie	z_1	z_2	z_3	z_4	z_5	z_6	z_7	z_8	z_9
t_i	3	5	6	10	11	14	15	18	20

Przydzielając zadania od najkrótszego do najdłuższego, każde do pierwszego wolnego procesora, otrzymujemy następujące uszeregowanie:



Procesory kończą pracę odpowiednio w chwilach 28 (P_1), 34 (P_2) i 40 (P_3).

Rozwiązanie. Przydzielaj zadania do pierwszego wolnego procesora w kolejności od najkrótszego do najdłuższego.

Wniosek 1. Nasze algorytmy szeregowania zadań (na jednym oraz na wielu procesorach) są poprawne – minimalizują średni czas zakończenia.

Poprawność wynika z tego samego argumentu wymiany: gdyby w optymalnym uszeregowaniu dwa zadania przydzielone do tego samego procesora były ułożone niezgodnie z porządkiem rosnącym czasów, ich zamiana nie zwiększyłaby średniego czasu zakończenia. Dlatego strategia zachłanna – najkrótsze zadanie do najwcześniej zwalnającego się procesora – daje rozwiązanie optymalne.

Notatki z wykładu: Problem sumy podzbiorów
NP-zupełność i w pełni wielomianowy schemat aproksymacyjny

15 kwietnia 2026

1 Problem sumy podzbiorów

Dany jest zbiór liczb naturalnych $S = \{x_1, \dots, x_n\} \subseteq \mathbb{N}$ oraz liczba $t \in \mathbb{N}$.

Definicja 1 (wersja decyzyjna). Czy istnieje podzbiór $S' \subseteq S$ taki, że $\sum_{x_i \in S'} x_i = t$?

Definicja 2 (wersja optymalizacyjna). Znaleźć podzbiór $S' \subseteq S$ taki, że suma $\sum_{x_i \in S'} x_i$ jest *największa możliwa*, ale nie przekracza t .

Oznaczenia. Dla zbioru $P \subseteq \mathbb{N}$ i liczby $x \in \mathbb{N}$ przyjmujemy

$$P + x = \{s + x : s \in P\}.$$

Na przykład dla $P = \{1, 2, 3, 6\}$ oraz $x = 2$ mamy $P + x = \{3, 4, 5, 8\}$.

Analogicznie dla *listy* L liczb naturalnych i $x \in \mathbb{N}$ przez $L + x$ oznaczamy listę powstałą z L przez dodanie x do każdego elementu. Na przykład dla $L = (3, 5, 8)$ i $x = 3$ mamy $L + x = (6, 8, 11)$.

2 Dokładny algorytm

Lemat 1. Niech $P_0 = \{0\}$ oraz $P_i = P_{i-1} \cup (P_{i-1} + x_i)$ dla $i \in [n]$. Wtedy zbiór P_n składa się ze wszystkich możliwych sum podzbiorów zbioru S .

Dowód. Pokażemy przez indukcję po i , że P_i składa się ze wszystkich możliwych sum elementów podzbiorów zbioru $\{x_1, x_2, \dots, x_i\}$.

Baza ($i = 0$). $P_0 = \{0\}$, a jedyny podzbiór $\emptyset$ ma sumę $\sum_{x_i \in \emptyset} x_i = 0$. ✓

Krok indukcyjny. Załóżmy, że P_{i-1} składa się ze wszystkich możliwych sum elementów podzbiorów zbioru $\{x_1, \dots, x_{i-1}\}$. Niech $S' \subseteq \{x_1, \dots, x_i\}$.

- Jeśli $x_i \notin S'$, to $S' \subseteq \{x_1, \dots, x_{i-1}\}$, więc

$$\sum_{x_k \in S'} x_k \in P_{i-1} \subseteq P_i.$$

- Jeśli $x_i \in S'$, to $S' \setminus \{x_i\} \subseteq \{x_1, \dots, x_{i-1}\}$, więc

$$\sum_{x_k \in S'} x_k = \left(\sum_{x_k \in S' \setminus \{x_i\}} x_k \right) + x_i \in P_{i-1} + x_i \subseteq P_i.$$

Zatem P_i zawiera wszystkie możliwe sumy elementów z $\{x_1, \dots, x_i\}$. Nie zawiera też żadnych innych elementów, bo P_{i-1} to sumy podzbiorów $\{x_1, \dots, x_{i-1}\}$, a $P_{i-1} + x_i$ to sumy podzbiorów zawierających x_i . Dla $i = n$ otrzymujemy tezę. $\square$

Zakładamy, że dysponujemy procedurą MERGE-LISTS, która na wejściu bierze dwie *posortowane* listy liczb naturalnych L, L' i zwraca posortowaną listę elementów z $L \cup L'$ z usuniętymi duplikatami. Jej złożoność wynosi

$$\Theta(|L| + |L'|).$$

Algorithm 1 Extract-Subset-Sum(S, t)

```

1:  $n \leftarrow |S|$ 
2:  $L_0 \leftarrow (0)$ 
3: for  $i \leftarrow 1$  to  $n$  do
4:    $L_i \leftarrow \text{MERGE-LISTS}(L_{i-1}, L_{i-1} + x_i)$ 
5:   usuń z  $L_i$  wszystkie liczby większe od  $t$ 
6: end for
7: return największy element listy  $L_n$ 

```

Poprawność wyniku bezpośrednio z Lematu 1.

3 Analiza złożoności

Długość listy L_i może być **wykładnicza** względem i — nawet 2^i .

Dla instancji $I = (S, t)$ rozmiar wejścia wynosi

$$|I| = \Theta\left(\log t + \sum_{i=1}^n \log x_i\right).$$

Jeśli t oraz x_i są liczbami n -cyfrowymi, to $\log t, \log x_i = \Theta(n)$, więc

$$|I| = \Theta(n + n^2) = \Theta(n^2).$$

Skoro lista L_n może mieć długość 2^n , algorytm EXTRACT-SUBSET-SUM jest *wykładniczy*.

Jeśli natomiast t jest ograniczone przez funkcję wielomianową od $n = |S|$, czyli $t = O(n^p)$ dla pewnego $p \in \mathbb{N}$, to długości list L_i są ograniczone przez t i w tym przypadku algorytm działa w czasie wielomianowym.

4 NP-zupełność problemu sumy podzbiorów

Twierdzenie 1. *Problem sumy podzbiorów jest NP-zupełny.*

Dowód.

1. Problem należy do NP. Mając daną dla (S, t) odpowiedź $S' \subseteq S$, możemy w czasie wielomianowym sprawdzić, czy $\sum_{x_i \in S'} x_i = t$.

2. Redukcja 3-SAT $\leq_p$ Subset-Sum. Niech Φ będzie formułą 3-SAT w postaci CNF, ze zmiennymi $x_1, x_2, \dots, x_n$ i klauzulami $C_1, C_2, \dots, C_m$. Bez straty ogólności zakładamy, że:

- żadna klauzula nie zawiera jednocześnie zmiennej i jej zaprzeczenia (w przeciwnym razie klauzula jest trywialnie spełniona);
- każda zmienna występuje w co najmniej jednej klauzuli.

Zbudujemy instancję (S, t) problemu SUBSET-SUM.

Konstrukcja. Wszystkie liczby mają $n + m$ cyfr. Pierwsze n cyfr etykietujemy zmiennymi, ostatnie m cyfr — klauzulami.

- Dla każdej zmiennej x_i tworzymy dwie liczby r_i oraz r'_i . Obie mają cyfrę 1 na pozycji zmiennej x_i i 0 na pozostałych pozycjach zmiennych. Ponadto:
 - jeśli literal x_i występuje w C_j , to r_i ma cyfrę 1 na pozycji C_j ;
 - jeśli literal $\overline{x_i}$ występuje w C_j , to r'_i ma cyfrę 1 na pozycji C_j .
- Dla każdej klauzuli C_j tworzymy dwie liczby uzupełniające: s_j z cyfrą 1 na pozycji C_j oraz s'_j z cyfrą 2 na pozycji C_j (0 na pozostałych pozycjach).
- Liczba docelowa t ma cyfrę 1 na każdej pozycji zmiennej oraz cyfrę 4 na każdej pozycji klauzuli:

$$t = \underbrace{11 \dots 1}_n \underbrace{44 \dots 4}_m.$$

Liczby r_i, r'_i, s_j, s'_j są parami różne (wynika to z konstrukcji i założeń o Φ). Suma cyfr na każdej pozycji wszystkich liczb wynosi co najwyżej 6, więc przy dodawaniu *nie występują przeniesienia* — każdą pozycję można analizować niezależnie.

Poprawność. ($\Rightarrow$) Jeśli Φ jest spełnialna, ustalmy wartościowanie spełniające. Do S' bierzemy r_i , gdy $x_i = 1$, oraz r'_i , gdy $x_i = 0$; wtedy każda pozycja zmiennej sumuje się do 1. Dla pozycji klauzuli C_j liczba wybranych liczb r/r' mających tam 1 jest równa liczbie spełnionych literalów w C_j , czyli należy do $\{1, 2, 3\}$. Dobierając liczby uzupełniające s_j (cyfra 1) i/lub s'_j (cyfra 2) dopełniamy każdą pozycję klauzuli do 4. Stąd suma S' jest równa t .

($\Leftarrow$) Jeśli istnieje $S' \subseteq S$ o sumie t , to dla każdej pozycji zmiennej dokładnie jedna z liczb r_i, r'_i należy do S' (bo tylko one mają tam niezerową cyfrę, a suma na tej pozycji to 1). Przyjmujemy $x_i = 1$, gdy $r_i \in S'$, oraz $x_i = 0$, gdy $r'_i \in S'$. Dla każdej pozycji klauzuli C_j liczby uzupełniające dają co najwyżej $1 + 2 = 3$, więc do osiągnięcia 4 potrzebna jest co najmniej jedna liczba r_i lub r'_i mająca 1 na pozycji C_j — czyli literal spełniający C_j . Zatem każda klauzula jest spełniona i Φ jest spełnialna.

Redukcja działa w czasie wielomianowym: konstruujemy $2(n + m)$ liczb o $n + m$ cyfrach, a ich utworzenie i posortowanie jest wielomianowe względem $n + m$. $\square$

4.1 Przykład redukcji

Rozważmy formułę o $n = 3$ zmiennych i $m = 4$ klauzulach:

$$C_1 = (x_1 \vee \overline{x_2} \vee \overline{x_3}), \quad C_2 = (\overline{x_1} \vee \overline{x_2} \vee \overline{x_3}), \quad C_3 = (\overline{x_1} \vee \overline{x_2} \vee x_3), \quad C_4 = (x_1 \vee x_2 \vee x_3),$$

$$\Phi = C_1 \wedge C_2 \wedge C_3 \wedge C_4.$$

Odpowiadająca instancja SUBSET-SUM (liczby $(n + m) = 7$ -cyfrowe):

	x_1	x_2	x_3	C_1	C_2	C_3	C_4
r_1	1	0	0	1	0	0	1
r'_1	1	0	0	0	1	1	0
r_2	0	1	0	0	0	0	1
r'_2	0	1	0	1	1	1	0
r_3	0	0	1	0	0	1	1
r'_3	0	0	1	1	1	0	0
s_1	0	0	0	1	0	0	0
s'_1	0	0	0	2	0	0	0
s_2	0	0	0	0	1	0	0
s'_2	0	0	0	0	2	0	0
s_3	0	0	0	0	0	1	0
s'_3	0	0	0	0	0	2	0
s_4	0	0	0	0	0	0	1
s'_4	0	0	0	0	0	0	2
t	1	1	1	4	4	4	4

Dla wartościowania $x_1 = 1$, $x_2 = 0$, $x_3 = 0$ (spełniającego Φ) wybieramy r_1, r'_2, r'_3 , których suma na pozycjach klauzul to $(3, 2, 1, 1)$. Dopełniając liczbami uzupełniającymi $s_1, s'_2, s_3, s'_3, s_4, s'_4$ otrzymujemy na każdej pozycji klauzuli wartość 4, więc

$$S' = \{r_1, r'_2, r'_3, s_1, s'_2, s_3, s'_3, s_4, s'_4\} \quad \text{ma sumę } t.$$

5 W pełni wielomianowy schemat aproksymacyjny

Pokażemy w pełni wielomianowy schemat aproksymacyjny (FPTAS) dla wersji optymalizacyjnej.

Pomysł. Będziemy *skracać* listy L_i . Niech $0 < \delta < 1$. Z listy L tworzymy krótszą listę L' tak, by dla każdego elementu y usuniętego z L istniał w L' element z leżący blisko niego od dołu, dokładniej:

$$\frac{y}{1+\delta} \leq z \leq y.$$

Przykład. Dla $L = (15, 16, 17, 25, 27, 28, 35, 36, 37)$ oraz $\delta = 0,1$ przycinanie daje

$$L' = (15, 17, 25, 28, 35).$$

(Np. 16 zostaje pochłonięte przez 15, bo $16 \leq 15 \cdot 1,1 = 16,5$; element 17 zostaje, bo $17 > 16,5$.)

Procedura TRIM realizuje ten pomysł dla listy $L = (y_1, \dots, y_m)$ posortowanej rosnąco i parametru $0 < \delta < 1$:

Czas działania procedury TRIM jest liniowy względem $|L|$.

Schemat aproksymacyjny powstaje z dokładnego algorytmu przez przycinanie list parametrem $\delta = \varepsilon/(2n)$ w każdej iteracji:

Algorithm 2 Trim(L, δ)

```
1:  $m \leftarrow |L|$ 
2:  $L' \leftarrow (y_1)$ 
3:  $last \leftarrow y_1$ 
4: for  $i \leftarrow 2$  to  $m$  do
5:   if  $y_i > last \cdot (1 + \delta)$  then
6:     dołącz  $y_i$  na koniec  $L'$ 
7:      $last \leftarrow y_i$ 
8:   end if
9: end for
10: return  $L'$ 
```

Algorithm 3 Approx-Subset-Sum(S, t, ε)

```
1:  $n \leftarrow |S|$ 
2:  $L_0 \leftarrow (0)$ 
3: for  $i \leftarrow 1$  to  $n$  do
4:    $L_i \leftarrow \text{MERGE-LISTS}(L_{i-1}, L_{i-1} + x_i)$ 
5:    $L_i \leftarrow \text{TRIM}(L_i, \varepsilon/(2n))$ 
6:   usuń z  $L_i$  wszystkie liczby większe od  $t$ 
7: end for
8: return największy element listy  $L_n$ 
```

Algorytmy Zachłanne – Wykład

22 kwietnia 2026

1 Matroidy

Definicja 1. Matroid to para $M = (S, \mathcal{A})$ taka, że:

1. S – niepusty zbiór skończony,
2. $\mathcal{A} \subseteq 2^S$ i $\mathcal{A} \neq \emptyset$ – czyli $\mathcal{A}$ jest niepustą rodziną podzbiorów S ,
3. $\forall A, B \subseteq S \quad A \in \mathcal{A} \wedge B \subseteq A \Rightarrow B \in \mathcal{A}$,
4. $\forall A, B \subseteq S \quad A, B \in \mathcal{A} \wedge |B| > |A| \Rightarrow \exists x \in B \setminus A$ takie, że $A \cup \{x\} \in \mathcal{A}$ (własność wymiany).

$\mathcal{A}$ – rodzina zbiorów niezależnych matroidu M . Elementy $\mathcal{A}$ – zbiory niezależne matroidu M .

1.1 Przykłady

Przykład 1. A – macierz, S – zbiór wierszy A ,

$$\mathcal{A} = \{R \subseteq S : R \text{ tworzy zbiór liniowo niezależny}\}.$$

$(S, \mathcal{A})$ – matroid.

Przykład 2. $G = (V, E)$ – graf,

$$\mathcal{A} = \{F \subseteq E : G[F] \text{ jest acykliczny}\}.$$

$(E, \mathcal{A})$ – matroid zwany **matroidem grafowym**.

Z definicji $\emptyset \in \mathcal{A}$.

1.2 Własności

Stwierdzenie 1. $M = (S, \mathcal{A})$. Wszystkie maksymalne zbiory niezależne M mają tę samą liczbę.

Dowód. Niech $A, B \in \mathcal{A}$ będą maksymalnymi zbiorami niezależnymi M . Przypuśćmy, że $|B| > |A|$. Z własności 4 istnieje $x \in B \setminus A$ takie, że $A \cup \{x\} \in \mathcal{A}$ – sprzeczność z maksymalnością A . $\square$

2 Problem optymalizacji na matroidzie

$M = (S, \mathcal{A})$ – matroid, $w: S \rightarrow \mathbb{R}_+$ – funkcja wagi.

Dla $A \subseteq S$ niech

$$w(A) = \sum_{x \in A} w(x) \rightarrow \text{waga } A.$$

Problem: Znaleźć zbiór niezależny o maksymalnej wadze w M .

Niech $S = \{x_1, x_2, \dots, x_n\}$.

2.1 Algorytm Greedy

Algorithm 1 Greedy(M, w)

```

1:  $n \leftarrow |S|$ 
2:  $A \leftarrow \emptyset$   $\{A - \text{rozwiązanie}\}$ 
3: posortuj  $S$  w porządku nierosnącym względem wagi, tak żeby  $w(x_1) \geq w(x_2) \geq \dots \geq w(x_n)$ 
4: for  $i \leftarrow 1$  to  $n$  do
5:   if  $A \cup \{x_i\} \in \mathcal{A}$  then
6:      $A \leftarrow A \cup \{x_i\}$ 
7:   end if
8: end for
9: return  $A$ 

```

2.2 Poprawność algorytmu

Twierdzenie 1 (Rado, Edmonds). Algorytm Greedy znajduje zbiór niezależny o maksymalnej wadze.

Dowód. Niech $A = \{x_{i_1}, x_{i_2}, \dots, x_{i_k}\}$ będzie zbiorem zwróconym przez algorytm i $w(x_{i_1}) \geq w(x_{i_2}) \geq \dots \geq w(x_{i_k})$.

Krok 1. Najpierw pokażemy, że A jest maksymalnym zbiorem niezależnym.

Przypuśćmy, że istnieje $x_j \in S \setminus A$ takie, że $A \cup \{x_j\} \in \mathcal{A}$. Wtedy $j < i_k$, bo w przeciwnym przypadku algorytm dodałby x_j do A jako $(k+1)$ -szy element.

Niech ℓ będzie najmniejszą liczbą taką, że $j < i_\ell$. Wtedy

$$w(x_{i_1}) \geq w(x_{i_2}) \geq \dots \geq w(x_{i_{\ell-1}}) \geq w(x_j) \geq w(x_{i_\ell}).$$

Ale wtedy algorytm dodałby x_j zamiast x_{i_ℓ} jako ℓ -ty element dodany do A – sprzeczność.

Zatem A jest maksymalnym zbiorem niezależnym.

Krok 2. Rozważmy zbiór niezależny $B = \{y_1, y_2, \dots, y_m\} \in \mathcal{A}$ i załóżmy, że $w(y_1) \geq w(y_2) \geq \dots \geq w(y_m)$.

Mamy $m \leq k$, bo A jest maksymalnym zbiorem niezależnym (Stwierdzenie 1).

Pokażemy, że $w(B) \leq w(A)$.

Pokażemy, że dla każdego $j \leq m$: $w(y_j) \leq w(x_{i_j})$.

Przypuśćmy, że $w(y_j) > w(x_{i_j})$. Rozważmy zbiory niezależne

$$A_{j-1} = \{x_{i_1}, \dots, x_{i_{j-1}}\} \subseteq A, \quad B_j = \{y_1, \dots, y_j\} \subseteq B.$$

$j = |B_j| > |A_{j-1}| = j - 1$, więc z własności 4 istnieje $y_s \in B_j \setminus A_{j-1}$ takie, że $A_{j-1} \cup \{y_s\} \in \mathcal{A}$.

Mamy $w(y_s) \geq w(y_j) > w(x_{i_j})$, i więc istnieje $t \leq j$ takie, że

$$w(x_{i_1}) \geq w(x_{i_2}) \geq \dots \geq w(x_{i_{t-1}}) \geq w(y_s) > w(x_{i_t}).$$

Z własności 3: $\{x_{i_1}, x_{i_2}, \dots, x_{i_{t-1}}, y_s\} \subseteq A_{j-1} \cup \{y_s\}$ jest zbiorem niezależnym.

Ale wtedy algorytm dodałby y_s zamiast x_{i_t} jako t -ty element dodany do A – sprzeczność.

Zatem

$$w(y_1) \leq w(x_{i_1}), \quad w(y_2) \leq w(x_{i_2}), \quad \dots, \quad w(y_m) \leq w(x_{i_m}),$$

a więc

$$w(B) = w(y_1) + \dots + w(y_m) \leq w(x_{i_1}) + \dots + w(x_{i_m}) \leq w(A). \quad \square$$

2.3 Złożoność czasowa algorytmu Greedy

Niech $n = |S|$.

linia 1	$O(n)$
linia 2	$O(1)$
linia 3	$O(n \log n)$
liczba iteracji pętli w linii 4	wynosi n
linia 5	$f(n)$ – czas sprawdzenia warunku z linii 5
linia 6	$O(1)$
linia 7	$O(1)$

Całkowita złożoność wynosi $O(n \log n + n \cdot f(n))$.

3 Problem szeregowania zadań

$S = \{z_1, z_2, \dots, z_n\}$ – zbiór zadań o jednostkowym czasie wykonania.

$d_1, d_2, \dots, d_n$ – deadliny, $d_i \in \{1, \dots, n\}$ dla $i \in [n]$.

$w_1, w_2, \dots, w_n$ – kary, $w_i \geq 0$ dla $i \in [n]$.

Płacimy karę w_i za zadanie z_i jeśli nie jest skończone w momencie d_i .

Problem: Znaleźć kolejność wykonania zadań minimalizującą sumę kar.

Założmy, że mamy pewien harmonogram. Zadanie z_i nazywamy **terminowym** w tym harmonogramie, jeśli jest zakończone do chwili d_i . W przeciwnym razie nazywamy je **spóźnionym**.

Nie płacimy kar za zadania terminowe, płacimy kary za zadania spóźnione.

Lemat 1. Każdy harmonogram można przetransformować na harmonogram z taką samą sumą kar, w którym wszystkie zadania terminowe poprzedzają wszystkie zadania spóźnione oraz zadania terminowe są posortowane w kolejności niemalejącej względem deadlinów.

Dowód. **1.** Niech z_i będzie zadaniem spóźnionym ukończonym w chwili $l_i > d_i$ i niech z_j będzie zadaniem terminowym ukończonym w chwili $l_j \leq d_j$, i niech z_i będzie przed z_j w harmonogramie, tzn. $l_i < l_j$.

Zamieniamy miejscami z_i i z_j . Po zamianie:

- z_i jest ukończone w chwili $l_j > l_i \geq d_i$, i więc nadal jest spóźnione.
- z_j jest ukończone w chwili $l_i < l_j \leq d_j$, więc nadal jest terminowe.

Po pewnej liczbie takich zamian otrzymujemy harmonogram o tej samej sumie kar, w którym wszystkie zadania terminowe poprzedzają wszystkie zadania spóźnione.

2. Niech z_i, z_j będą zadaniami terminowymi takimi, że z_i jest ukończone w chwili $l_i \leq d_i$, z_j jest ukończone w chwili $l_j \leq d_j$, $l_i < l_j$, ale $d_i > d_j$.

Zamieniamy miejscami z_i i z_j . Po zamianie:

- z_i jest ukończone w chwili $l_j \leq d_j < d_i$, więc z_i jest nadal terminowe.
- z_j jest ukończone w chwili $l_i < l_j \leq d_j$, więc z_j jest nadal terminowe.

Po pewnej liczbie takich zamian otrzymujemy harmonogram spełniający warunki lematu. $\square$

Notatki z wykładu: Algorytmy zachłanne

Szeregowanie zadań z karami oraz algorytmy aproksymacyjne

29 kwietnia 2026

Część I — Algorytmy zachłanne (kontynuacja)

1 Problem szeregowania zadań

Dany jest zbiór zadań o jednostkowym czasie wykonania

$$S = \{z_1, z_2, \dots, z_n\}.$$

Każde zadanie z_i ma **termin** (deadline) d_i oraz **karę** za opóźnienie w_i :

$$\forall i \in [n] \ d_i \in [n], \quad \forall i \in [n] \ w_i \in \mathbb{R}^+.$$

Karę w_i płacimy za zadanie z_i wtedy, gdy *nie* wyrobimy się z jego wykonaniem do chwili d_i .

Harmonogram to ustalona kolejność wykonywania zadań. Zadanie nazywamy **terminowym**, jeśli kończy się przed swoim terminem; w przeciwnym przypadku jest **spóźnione**.

Celem jest znalezienie harmonogramu minimalizującego łączną karę.

2 Zbiory wolne od kar

Definicja 1 (zbiór wolny od kar). Zbiór zadań $A \subseteq S$ nazywamy **wolnym od kar**, jeśli istnieje harmonogram, w którym wszystkie zadania z A są terminowe.

Dla $t \in \{0, 1, \dots, n\}$ wprowadzamy oznaczenie

$$N_t(A) = |\{z_i \in A : d_i \leq t\}| \quad (\text{liczba zadań z } A \text{ o terminie nie późniejszym niż } t).$$

Lemat 1 (charakteryzacja zbiorów wolnych od kar). *Dla dowolnego zbioru $A \subseteq S$ następujące warunki są równoważne:*

- (1) A jest wolny od kar;
- (2) $N_t(A) \leq t$ dla każdego $t \in \{1, \dots, n\}$;
- (3) jeśli zadania z A ustawimy w kolejności niemalejącej według terminów, to żadne zadanie nie jest spóźnione.

Dowód. **(1) $\Rightarrow$ (2).** Przypuśćmy nie wprost, że istnieje t takie, że $N_t(A) > t$. Wtedy w A jest więcej niż t zadań o terminie $\leq t$. Przy jednostkowym czasie wykonania do chwili t można ukończyć co najwyżej t zadań, więc któreś z tych zadań musi być spóźnione — sprzeczność z tym, że A jest wolny od kar. Zatem $N_t(A) \leq t$ dla każdego t .

(2) $\Rightarrow$ (3). Ustawmy zadania niemalejąco według terminów i przypuśćmy nie wprost, że pewne zadanie z kończy się w chwili $\ell > d_z$ (jest spóźnione). Ponieważ kolejność jest niemalejąca, wszystkie ℓ zadań wykonanych do chwili ℓ ma termin $\leq d_z$. Stąd

$$N_{d_z}(A) \geq \ell > d_z,$$

co przeczy warunkowi (2). Zatem żadne zadanie nie jest spóźnione.

(3) $\Rightarrow$ (1). Oczywiście: harmonogram posortowany według terminów jest harmonogramem bez spóźnień, więc A jest wolny od kar. $\square$

2.1 Sprowadzenie do problemu zachłannego

Chcemy znaleźć zbiór A wolny od kar taki, że suma kar zadań spóźnionych

$$\sum_{z_i \notin A} w_i$$

jest najmniejsza możliwa. Ponieważ

$$\sum_{z_i \notin A} w_i = \sum_{z_i \in S} w_i - \underbrace{\sum_{z_i \in A} w_i}_{(*)}$$

a pierwszy składnik jest stały, minimalizacja kary jest *równoważna* maksymalizacji sumy $(*)$ wag zadań terminowych. Szukamy więc wolnego od kar zbioru A maksymalizującego $\sum_{z_i \in A} w_i$.

3 Struktura matroidu

Niech $\mathcal{A}$ będzie rodziną wszystkich zbiorów wolnych od kar.

Twierdzenie 1. Niech S będzie zbiorem zadań o jednostkowym czasie wykonania, a $\mathcal{A}$ rodziną zbiorów wolnych od kar. Wtedy $(S, \mathcal{A})$ jest **matroidem**.

Dowód. Sprawdzamy cztery warunki definicji matroidu.

(1) **Skończoność.** Zbiór S jest skończony — oczywiście.

(2) **Niepustość.** $\mathcal{A}$ jest niepustym podzbiorem zbioru potęgowego 2^S : dla każdego $i \in [n]$ pojedyncze zadanie $\{z_i\}$ jest wolne od kar (można je wykonać do chwili $d_i \geq 1$), więc $\{z_i\} \in \mathcal{A}$.

(3) **Dziedziczność.** Niech $A \in \mathcal{A}$ oraz $B \subseteq A$. Istnieje harmonogram, w którym wszystkie zadania z A są terminowe. Bierzemy ten harmonogram i usuwamy z niego wszystkie zadania nienależące do B ; otrzymujemy harmonogram dla B , w którym zadania nadal pozostają terminowe. Zatem $B \in \mathcal{A}$.

(4) **Własność wymiany.** Niech $A, B \in \mathcal{A}$ oraz $|B| > |A|$. Zdefiniujemy

$$k = \text{największa liczba } t \in \{0, 1, \dots, n\} \text{ taka, że } N_t(B) \leq N_t(A).$$

Liczba k jest dobrze określona, bo $N_0(B) = 0 = N_0(A)$, więc warunek zachodzi dla $t = 0$. Podstawiając $t = n$ dostajemy

$$N_n(B) = |B| > |A| = N_n(A),$$

więc warunek $N_t(B) \leq N_t(A)$ nie zachodzi dla $t = n$, skąd $k < n$. Z definicji k mamy ponadto

$$N_k(B) \leq N_k(A) \quad \text{oraz} \quad N_{k+1}(B) > N_{k+1}(A).$$

Stąd istnieje co najmniej jedno zadanie $z \in B \setminus A$ o terminie równym $k + 1$. Połóżmy

$$A' = A \cup \{z\}.$$

Pokażemy, używając warunku (2) Lematu 1, że A' jest wolny od kar:

- dla $t \in \{1, \dots, k\}$: zadanie z ma termin $k + 1 > t$, więc nie jest liczone, i

$$N_t(A') = N_t(A) \leq t \quad (\text{bo } A \text{ jest wolny od kar});$$

- dla $t \in \{k + 1, \dots, n\}$: z definicji k mamy $N_t(A) < N_t(B)$, czyli $N_t(A) + 1 \leq N_t(B)$, zatem

$$N_t(A') = N_t(A) + 1 \leq N_t(B) \leq t \quad (\text{bo } B \text{ jest wolny od kar}).$$

W obu przypadkach $N_t(A') \leq t$, więc A' jest wolny od kar, czyli $A \cup \{z\} \in \mathcal{A}$. To kończy dowód własności wymiany, a tym samym dowód, że $(S, \mathcal{A})$ jest matroidem. $\square$

Wniosek 1 (twierdzenie Rado–Edmondsa). *Ponieważ $(S, \mathcal{A})$ jest matroidem, algorytm zachłanny zastosowany do tego problemu (z wagami w_i jako kluczem) zwraca rozwiązanie optymalne.*

Algorytm. Sortujemy zadania nierosnąco według kar w_i i kolejno próbujemy dodać każde zadanie do budowanego zbioru A , zachowując je wtedy i tylko wtedy, gdy A pozostaje wolny od kar.

Złożoność. Sortowanie kosztuje $O(n \log n)$, a sprawdzanie warunku „wolny od kar” dla kolejnych zadań — $O(n)$ na zadanie, czyli $O(n^2)$ łącznie. Stąd

$$O(n \log n + n \cdot n) = O(n^2).$$

4 Przykład

Rozważmy $n = 8$ zadań o następujących terminach i karach:¹

	z_1	z_2	z_3	z_4	z_5	z_6	z_7	z_8
d_i	3	2	2	3	1	4	6	8
w_i	80	70	60	50	40	30	20	10

Przetwarzamy zadania w kolejności nierosnących kar (czyli $z_1, z_2, \dots, z_8$) i dla każdego sprawdzamy, czy po dodaniu zbiór pozostaje wolny od kar (warunek $N_t \leq t$):

¹W oryginalnych notatkach termin d_3 był nieczytelny; przyjęto $d_3 = 2$, co jest spójne z końcowym harmonogramem zapisanym w notatkach. Warto to zweryfikować.

krok	rozważane zadanie	bieżący zbiór A	decyzja
1	z_1 ($d=3$)	$\{z_1\}$	przyjmij
2	z_2 ($d=2$)	$\{z_1, z_2\}$	przyjmij
3	z_3 ($d=2$)	$\{z_1, z_2, z_3\}$	przyjmij
4	z_4 ($d=3$)	—	odrzuć (N_3 wzrósłby do $4 > 3$)
5	z_5 ($d=1$)	—	odrzuć (N_2 wzrósłby do $3 > 2$)
6	z_6 ($d=4$)	$\{z_1, z_2, z_3, z_6\}$	przyjmij
7	z_7 ($d=6$)	$\{z_1, z_2, z_3, z_6, z_7\}$	przyjmij
8	z_8 ($d=8$)	$\{z_1, z_2, z_3, z_6, z_7, z_8\}$	przyjmij

Wynikowy zbiór terminowy to $A = \{z_1, z_2, z_3, z_6, z_7, z_8\}$, a zadaniami spóźnionymi są z_4 i z_5 . Suma kar wynosi

$$w_4 + w_5 = 50 + 40 = 90.$$

Ostateczny harmonogram (zadania terminowe według terminów, a na końcu spóźnione):

$$(z_2, z_3, z_1, z_6, z_7, z_8 \mid z_4, z_5).$$

Część II — Algorytmy aproksymacyjne

5 Algorytmy aproksymacyjne

Problemy, dla których znane są algorytmy wielomianowe, nazywamy *łatwymi*. Tam, gdzie takie algorytmy nie są znane, mówimy o problemach *trudnych*. Dla tych ostatnich **algorytm aproksymacyjny** znajduje rozwiązanie *prawie* optymalne.

Niech Π będzie problemem optymalizacyjnym (maksymalizacji albo minimalizacji pewnej funkcji celu f). Wprowadzamy oznaczenia:

- D_Π — zbiór instancji problemu Π , a $I \in D_\Pi$ — pojedyncza instancja;
- $S(I)$ — zbiór rozwiązań dopuszczalnych instancji I ;
- $f : \bigcup_{I \in D_\Pi} S(I) \rightarrow \mathbb{R}$ — funkcja celu;
- $c(I) > 0$ — wartość rozwiązania zwróconego przez algorytm dla instancji I ;
- $c^*(I)$ — wartość rozwiązania optymalnego dla instancji I .

Definicja 2 (współczynnik aproksymacji). Mówimy, że algorytm A ma **współczynnik aproksymacji** $\rho(n)$, jeśli dla każdej instancji $I \in D_\Pi$ rozmiaru n zachodzi

$$\max \left\{ \frac{c(I)}{c^*(I)}, \frac{c^*(I)}{c(I)} \right\} \leq \rho(n).$$

Wtedy A nazywamy algorytmem $\rho(n)$ -aproksymacyjnym.

Konwencja jest tak dobrana, że $\rho(n) \geq 1$ niezależnie od typu problemu:

$$\text{maksymalizacja: } 0 < c(I) \leq c^*(I), \quad \text{minimalizacja: } 0 < c^*(I) \leq c(I).$$

W każdym przypadku jeden z dwóch ilorazów jest równy 1, a drugi ≥ 1 , więc ich maksimum jest ≥ 1 . Przypadek $\rho(n) = 1$ oznacza algorytm *dokładny*.

Definiujemy też współczynnik asymptotyczny

$$\rho = \sup_{n \in \mathbb{N}} \rho(n);$$

mówimy wtedy, że współczynnik aproksymacji algorytmu A wynosi ρ , i nazywamy A algorytmem ρ -aproksymacyjnym.

6 Problem pokrycia wierzchołkowego (Vertex Cover)

Niech $G = (V, E)$ będzie grafem.

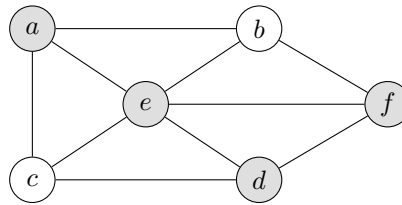
Definicja 3 (pokrycie wierzchołkowe). **Pokryciem wierzchołkowym** grafu G nazywamy zbiór wierzchołków $W \subseteq V$ taki, że każda krawędź ma co najmniej jeden koniec w W :

$$\forall e \in E : e \cap W \neq \emptyset.$$

Problem. Znaleźć pokrycie wierzchołkowe grafu G o *minimalnej* liczbie wierzchołków.

Przykład

W poniższym grafie zbiór $\{a, d, e, f\}$ (wierzchołki zacienione) jest pokryciem wierzchołkowym — każda krawędź ma w nim swój koniec:



Algorytm aproksymacyjny

Algorithm 1 Approx-Vertex-Cover(G)

```

1:  $C \leftarrow \emptyset$  {budowane pokrycie}
2:  $F \leftarrow E(G)$  {zbiór niepokrytych krawędzi}
3: while  $F \neq \emptyset$  do
4:   wybierz dowolną krawędź  $(u, v) \in F$ 
5:    $C \leftarrow C \cup \{u, v\}$ 
6:   usuń z  $F$  wszystkie krawędzie incydentne do  $u$  lub do  $v$ 
7: end while
8: return  $C$ 

```

Przykład działania

Dla grafu z rysunku powyżej kolejne wybory krawędzi mogą dawać:

$$(a, b) \Rightarrow C = \{a, b\}, \quad (c, d) \Rightarrow C = \{a, b, c, d\}, \quad (e, f) \Rightarrow C = \{a, b, c, d, e, f\}.$$

Algorytm zwrócił pokrycie o 6 wierzchołkach (zależnie od wyborów może być mniejsze).

Poprawność

Algorytm zwraca pokrycie wierzchołkowe. W każdym obrocie pętli wybieramy niepokrytą krawędź (u, v) i dodajemy do C oba jej końce, po czym usuwamy z F wszystkie krawędzie incydentne do u lub v . Tym samym w każdym obrocie liczność F maleje o co najmniej 1, a pętla obraca się, dopóki F nie stanie się puste. Zatem każda krawędź zostaje w pewnym momencie pokryta (ma koniec w C), więc C jest pokryciem wierzchołkowym grafu G .

Algorytmy zaawansowane

Algorytmy aproksymacyjne: pokrycie wierzchołkowe, pokrycie zbioru
oraz schematy aproksymacyjne

Wykład — 5 maja 2026

1 Pokrycie wierzchołkowe i algorytm AVC

Problem

Niech $G = (V, E)$ będzie grafem nieskierowanym. *Pokryciem wierzchołkowym* (ang. *vertex cover*) grafu G nazywamy taki zbiór $W \subseteq V$, że

$$\forall e \in E \quad e \cap W \neq \emptyset,$$

czyli każda krawędź ma co najmniej jeden koniec w W .

Instancja: graf $G = (V, E)$.

Problem: znaleźć pokrycie wierzchołkowe (VC) grafu G o minimalnej liczności.

Problem znajdowania minimalnego pokrycia wierzchołkowego jest NP-trudny, dlatego rozważamy algorytm aproksymacyjny AVC (*Approximate Vertex Cover*).

Algorytm

Algorytm 1 AVC(G)

1: $C \leftarrow \emptyset$	$\triangleright C$ — budowane rozwiązanie
2: $F \leftarrow E$	$\triangleright F$ — zbiór jeszcze nie pokrytych krawędzi
3: dopóki $F \neq \emptyset$ wykonuj	
4: wybierz dowolną krawędź $uv \in F$	
5: $C \leftarrow C \cup \{u, v\}$	
6: usuń z F wszystkie krawędzie incydentne z u lub v	
7: koniec dopóki	
8: zwróć C	

Złożoność czasowa

Niech $n = |V(G)|$ oraz $m = |E(G)|$.

- linia 1: $O(1)$,
- linia 2: $O(m)$,
- pętlę z wierszy 3–6 można zaimplementować tak, aby działała w czasie $O(m)$,
- linia 7: $O(1)$.

Łączna złożoność czasowa AVC wynosi $O(m)$.

Współczynnik aproksymacji

Twierdzenie 1.1. *AVC jest algorytmem 2-aproksymacyjnym.*

Dowód. Niech G będzie grafem wejściowym. Oznaczmy:

- $c^*(G)$ — najmniejszą liczbę wierzchołków w pokryciu wierzchołkowym G (rozmiar pokrycia optymalnego),
- $c(G)$ — liczbę wierzchołków w pokryciu wierzchołkowym znalezionym przez AVC.

Niech B będzie zbiorem krawędzi wybranych przez algorytm w linii 4. Zbiór B jest *skojarzeniem* (czyli zbiorem parami rozłącznych krawędzi): po wybraniu krawędzi uv algorytm usuwa z F wszystkie krawędzie incydentne z u lub v , więc żadna kolejna wybrana krawędź nie dzieli końca z uv .

Aby pokryć krawędzie z B , potrzeba co najmniej $|B|$ wierzchołków (każda z tych parami rozłącznych krawędzi wymaga osobnego wierzchołka pokrycia). Stąd

$$c^*(G) \geq |B|.$$

Z drugiej strony każda wybrana krawędź wnosi do C dwa wierzchołki, więc

$$c(G) = 2|B|.$$

Łącząc obie nierówności,

$$c(G) = 2|B| \leq 2c^*(G) \implies \frac{c(G)}{c^*(G)} \leq 2. \quad \square$$

2 Problem pokrycia zbioru i algorytm zachłanny GSC

Problem

Niech X będzie zbiorem skończonym, $|X| = n$, a $\mathcal{F} \subseteq 2^X$ rodziną podzbiorów zbioru X taką, że

$$\bigcup_{S \in \mathcal{F}} S = X$$

(każdy element X należy do co najmniej jednego zbioru z $\mathcal{F}$). Rodzinę $\mathcal{A} \subseteq \mathcal{F}$ nazywamy *pokryciem* zbioru X , jeśli

$$\bigcup_{S \in \mathcal{A}} S = X.$$

Instancja: para $(X, \mathcal{F})$, gdzie X jest zbiorem skończonym, $|X| = n$, oraz $\mathcal{F} \subseteq 2^X$, $\bigcup_{S \in \mathcal{F}} S = X$.

Problem: znaleźć pokrycie $\mathcal{A} \subseteq \mathcal{F}$ minimalizujące $|\mathcal{A}|$.

Przykład 2.1. Dla rodziny $\mathcal{F} = \{S_1, S_2, S_3, S_4, S_5, S_6\}$ pokrywającej X podrodzina $\{S_1, S_2, S_3\}$ może być pokryciem zbioru X .

Algorytm

Algorytm 2 GSC($X, \mathcal{F}$) — Greedy Set Cover

- 1: $U \leftarrow X$ $\triangleright U$ — zbiór jeszcze nie pokrytych elementów
 - 2: $\mathcal{A} \leftarrow \emptyset$ $\triangleright \mathcal{A}$ — budowane rozwiązanie
 - 3: **dopóki** $U \neq \emptyset$ **wykonuj**
 - 4: wybierz $S \in \mathcal{F}$ maksymalizujący $|S \cap U|$
 - 5: $\mathcal{A} \leftarrow \mathcal{A} \cup \{S\}$
 - 6: $U \leftarrow U - S$
 - 7: **koniec dopóki**
 - 8: **zwróć** $\mathcal{A}$
-

Poprawność

Pętla obraca się dopóki są jeszcze nie pokryte elementy, a w każdym obrocie pętli jakieś elementy zostają pokryte (wybrany S maksymalizuje $|S \cap U|$, a ponieważ $\mathcal{F}$ pokrywa X i $U \neq \emptyset$, to $|S \cap U| \geq 1$). Zatem algorytm zatrzymuje się i zwraca pokrycie X .

Złożoność czasowa

- linia 1: $O(n)$,
- linia 2: $O(1)$,
- liczba iteracji pętli z wierszy 3–6 jest $\leq |X|$ oraz $\leq |\mathcal{F}|$, więc jest $\leq \min\{|X|, |\mathcal{F}|\}$; wewnątrz pojedynczej iteracji:
 - linia 4: $O(|X| \cdot |\mathcal{F}|)$,
 - linia 5: $O(1)$,
 - linia 6: $O(|X|)$,co daje $O(|X| \cdot |\mathcal{F}|)$ na iterację,
- linia 7: $O(1)$.

Ostatecznie złożoność czasowa GSC wynosi

$$O(|X| \cdot |\mathcal{F}| \cdot \min\{|X|, |\mathcal{F}|\}).$$

Współczynnik aproksymacji

Przez H oznaczamy liczby harmoniczne:

$$H(0) = 0, \quad H(m) = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{m} \quad \text{dla } m \geq 1.$$

Twierdzenie 2.2. *GSC jest algorytmem $H(\max\{|S| : S \in \mathcal{F}\})$ -aproksymacyjnym.*

Dowód. Niech $(X, \mathcal{F})$ będzie instancją wejściową. Oznaczmy przez $\mathcal{A}^*$ optymalne pokrycie dla $(X, \mathcal{F})$, a przez $\mathcal{A}$ pokrycie znalezione przez GSC. Niech S_i będzie zbiorem wybranym przez algorytm w linii 4 w i -tej iteracji pętli.

Przypisanie kosztów. Każdemu elementowi $x \in S_i - (S_1 \cup \cdots \cup S_{i-1})$ (czyli elementowi pokrytemu po raz pierwszy w i -tej iteracji) przypisujemy koszt

$$c_x = \frac{1}{|S_i - (S_1 \cup \cdots \cup S_{i-1})|}.$$

Zbiór $S_i - (S_1 \cup \cdots \cup S_{i-1})$ to zbiór elementów pokrytych po raz pierwszy w i -tej iteracji, więc całkowity koszt przypisany podczas jednej iteracji wynosi 1. Ponieważ $|\mathcal{A}|$ jest liczbą iteracji pętli, mamy

$$|\mathcal{A}| = \sum_{x \in X} c_x.$$

Oszacowanie przez pokrycie optymalne. Skoro $\mathcal{A}^*$ jest pokryciem, każdy $x \in X$ występuje w co najmniej jednym zbiorze $S \in \mathcal{A}^*$, więc

$$\sum_{S \in \mathcal{A}^*} \sum_{x \in S} c_x \geq \sum_{x \in X} c_x = |\mathcal{A}|.$$

Pokażemy, że

$$\sum_{x \in S} c_x \leq H(|S|) \quad \text{dla każdego } S \in \mathcal{F}.$$

Niech $S \in \mathcal{F}$. Dla $i \in \{1, 2, \dots, |\mathcal{A}|\}$ niech

$$n_i = |S - (S_1 \cup \dots \cup S_i)|$$

(liczba elementów S , które nie są pokryte po i -tej iteracji) oraz $n_0 = |S|$. Niech k będzie najmniejsze takie, że $n_k = 0$ (wtedy S jest pokryty przez $S_1 \cup S_2 \cup \dots \cup S_k$). Oczywiście $n_{i-1} \geq n_i$ dla $i \in [k]$, a różnica $n_{i-1} - n_i$ to liczba elementów S pokrytych po raz pierwszy przez S_i . Zatem

$$\sum_{x \in S} c_x = \sum_{i=1}^k (n_{i-1} - n_i) \cdot \frac{1}{|S_i - (S_1 \cup \dots \cup S_{i-1})|}.$$

Na podstawie wyboru zachłannego w linii 4 mamy

$$|S_i - (S_1 \cup \dots \cup S_{i-1})| \geq |S - (S_1 \cup \dots \cup S_{i-1})| = n_{i-1} \quad \text{dla każdego } S \in \mathcal{F} - \{S_1, \dots, S_{i-1}\}.$$

Stąd

$$\begin{aligned} \sum_{x \in S} c_x &\leq \sum_{i=1}^k (n_{i-1} - n_i) \cdot \frac{1}{n_{i-1}} = \sum_{i=1}^k \sum_{j=n_i+1}^{n_{i-1}} \frac{1}{n_{i-1}} \leq \sum_{i=1}^k \sum_{j=n_i+1}^{n_{i-1}} \frac{1}{j} \\ &= \sum_{i=1}^k \left(\sum_{j=1}^{n_{i-1}} \frac{1}{j} - \sum_{j=1}^{n_i} \frac{1}{j} \right) = \sum_{i=1}^k (H(n_{i-1}) - H(n_i)) \\ &= H(n_0) - H(n_k) = H(|S|) - H(0) = H(|S|), \end{aligned}$$

gdzie wykorzystaliśmy teleskopowanie sumy.

Złożenie oszacowań.

$$\begin{aligned} |\mathcal{A}| = \sum_{x \in X} c_x &\leq \sum_{S \in \mathcal{A}^*} \sum_{x \in S} c_x \leq \sum_{S \in \mathcal{A}^*} H(|S|) \leq \sum_{S \in \mathcal{A}^*} \max\{H(|S|) : S \in \mathcal{A}^*\} \\ &= |\mathcal{A}^*| \cdot \max\{H(|S|) : S \in \mathcal{A}^*\} \leq |\mathcal{A}^*| \cdot \max\{H(|S|) : S \in \mathcal{F}\} \\ &= |\mathcal{A}^*| \cdot H(\max\{|S| : S \in \mathcal{F}\}), \end{aligned}$$

a więc

$$\frac{|\mathcal{A}|}{|\mathcal{A}^*|} \leq H(\max\{|S| : S \in \mathcal{F}\}). \quad \square$$

Wniosek 2.3. *GSC jest algorytmem $(\ln |X| + 1)$ -aproksymacyjnym.*

Dowód. Ponieważ $H(n) \leq \ln n + 1$, to

$$H(\max\{|S| : S \in \mathcal{F}\}) \leq H(|X|) \leq \ln |X| + 1. \quad \square$$

Pokrycie wierzchołkowe jako szczególny przypadek pokrycia zbioru

Pokrycie wierzchołkowe jest szczególnym przypadkiem problemu pokrycia zbioru. Dla $G = (V, E)$ przyjmujemy

$$X = E, \quad \mathcal{F} = \{E(v) : v \in V\}, \quad \text{gdzie } E(v) = \{e \in E : v \in e\},$$

przy czym $|E(v)| = d(v)$ (stopień wierzchołka v). Wtedy

$$\max\{|E(v)| : E(v) \in \mathcal{F}\} = \Delta(G),$$

więc GSC daje dla pokrycia wierzchołkowego współczynnik $H(\Delta(G))$. Ponieważ

$$H(3) = 1 + \frac{1}{2} + \frac{1}{3} = \frac{11}{6} < 2 < \frac{25}{12} = H(4),$$

dla grafów o maksymalnym stopniu ≤ 3 oszacowanie współczynnika aproksymacyjnego GSC jest lepsze niż oszacowanie dla AVC.

3 Schematy aproksymacyjne

Niech Π będzie problemem optymalizacyjnym (NP-trudnym).

Definicja 3.1 (schemat aproksymacyjny). Algorytm A jest *schematem aproksymacyjnym* dla Π , jeśli dla dowolnego wejścia (I, ε) , gdzie I jest instancją Π oraz $\varepsilon > 0$, algorytm A jest algorytmem $(1 + \varepsilon)$ -aproksymacyjnym.

Definicja 3.2 (wielomianowy schemat aproksymacyjny, PTAS). Schemat aproksymacyjny A nazywamy *wielomianowym schematem aproksymacyjnym* (PTAS), jeśli jego złożoność czasowa może być ograniczona funkcją wielomianową względem $|I|$ dla każdego ustalonego $\varepsilon > 0$.

Złożoność czasowa wielomianowego schematu aproksymacyjnego może wynosić np. $\Theta(2^{1/\varepsilon} n^2)$, gdzie $n = |I|$. Wtedy zwiększanie precyzji (czyli zmniejszanie ε) jest kosztowne czasowo.

Definicja 3.3 (w pełni wielomianowy schemat aproksymacyjny, FPTAS). Schemat aproksymacyjny A nazywamy *w pełni wielomianowym schematem aproksymacyjnym* (FPTAS), jeśli jego złożoność czasowa może być ograniczona funkcją wielomianową względem $|I|$ oraz $\frac{1}{\varepsilon}$.

Złożoność czasowa w pełni wielomianowego schematu aproksymacyjnego może wynosić np. $\Theta(n^2 \cdot \frac{1}{\varepsilon})$, gdzie $n = |I|$.

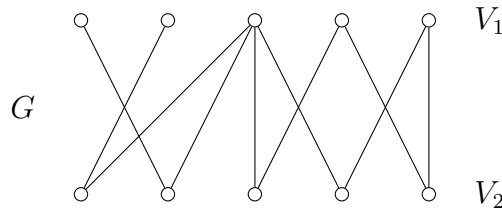
Skojarzenia w grafach dwudzielnych, 27.05

1 Podstawowe definicje

Niech

$$G = (V_1 \cup V_2, E)$$

będzie grafem dwudzielnym, gdzie V_1 oraz V_2 są rozłącznymi zbiorami wierzchołków, a każda krawędź łączy wierzchołek z V_1 z wierzchołkiem z V_2 .



Rysunek 1: Przykładowy graf dwudzielnny $G = (V_1 \cup V_2, E)$.

Definicja 1 (Skojarzenie). *Skojarzeniem w grafie G nazywamy zbiór krawędzi*

$$M \subseteq E,$$

taki że żadne dwie krawędzie z M nie mają wspólnego końca.

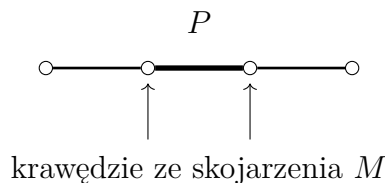
Wierzchołek nazywamy *wolnym* względem M , jeżeli nie jest incydentny z żadną krawędzią należącą do M .

Najliczniejsze Skojarzenie

M jest najliczniejszym skojarzeniem w $G \Leftrightarrow$ nie istnieje w G ścieżka powiększająca względem M .

Definicja 2 (Ścieżka powiększająca). *Ścieżką powiększającą względem skojarzenia M nazywamy ścieżkę prostą, która:*

- (i) *zaczyna się i kończy w wierzchołkach wolnych względem M ,*
- (ii) *ma krawędzie naprzemiennie nienależące i należące do M .*



krawędzie ze skojarzenia M

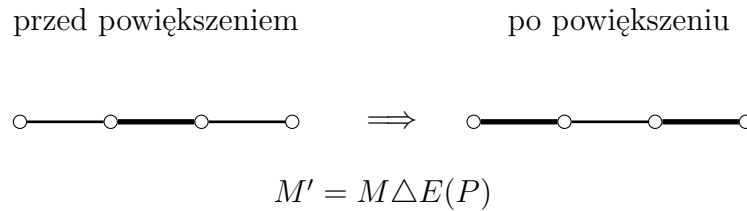
Rysunek 2: Ścieżka powiększająca względem skojarzenia M .

Jeżeli P jest ścieżką powiększającą, to nowe skojarzenie otrzymujemy jako różnicę symetryczną:

$$M' = M \triangle E(P).$$

Wtedy

$$|M'| = |M| + 1.$$



Rysunek 3: Powiększenie skojarzenia przez zmianę przynależności krawędzi ścieżki P .

2 Twierdzenie Berge'a

Twierdzenie 1 (Berge). *Skojarzenie M jest największym skojarzeniem w grafie G wtedy i tylko wtedy, gdy w grafie G nie istnieje ścieżka powiększająca względem M .*

3 Graf dwudzielny skierowany pomocniczy

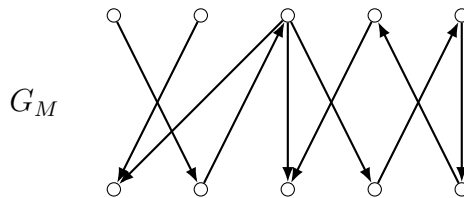
Dla danego skojarzenia M definiujemy graf skierowany

$$G_M = (V_1 \cup V_2, E_M),$$

gdzie

$$E_M = \{(u, v) : uv \in E \setminus M, u \in V_1, v \in V_2\} \cup \{(v, u) : uv \in M, u \in V_1, v \in V_2\}.$$

Krawędzie nienależące do skojarzenia są więc skierowane z V_1 do V_2 , natomiast krawędzie należące do skojarzenia są skierowane z V_2 do V_1 .



Rysunek 4: Graf skierowany pomocniczy G_M .

Ścieżka powiększająca względem M w grafie G odpowiada ścieżce skierowanej w grafie G_M zaczynającej się w wolnym wierzchołku z V_1 i kończącej się w wolnym wierzchołku z V_2 .

4 Algorytm znajdowania ścieżki powiększającej

Celem procedury APS jest znalezienie ścieżki powiększającej względem aktualnego skojarzenia M .

Algorithm 1 APS(G, M) – Augmenting Path Search

Require: Graf dwudzielny $G = (V_1 \cup V_2, E)$ oraz skojarzenie M w grafie G

Ensure: Ścieżka powiększająca P względem M albo NULL

1: $V'_1 \leftarrow$ zbiór wierzchołków wolnych z V_1

2: $V'_2 \leftarrow$ zbiór wierzchołków wolnych z V_2

3: Skonstruuj graf skierowany

$$G_M = (V_1 \cup V_2, E_M)$$

4: Znajdź ścieżkę skierowaną $\vec{P}$ z V'_1 do V'_2 w G_M

5: **if** $\vec{P}$ istnieje **then**

6: **return** P

▷ P jest ścieżką powiększającą

7: **else**

8: **return** NULL

9: **end if**

Lemat 1. Procedura APS zwraca ścieżkę $P \Leftrightarrow$ istnieje ścieżka powiększająca względem skojarzenia M w grafie G . Ponadto ścieżka P zwrócona przez APS jest ścieżką powiększającą względem M .

Dowód.

$\Rightarrow$ Załóżmy najpierw, że APS zwraca ścieżkę P . Oznacza to, że w grafie skierowanym G_M istnieje ścieżka skierowana $\vec{P}$ zaczynająca się w pewnym wierzchołku wolnym z V'_1 i kończąca się w pewnym wierzchołku wolnym z V'_2 .

Z definicji grafu G_M krawędzie spoza skojarzenia M są skierowane z V_1 do V_2 , natomiast krawędzie należące do M są skierowane z V_2 do V_1 . Zatem każda ścieżka skierowana w G_M odpowiada ścieżce w grafie G , której krawędzie występują naprzemiennie: najpierw krawędź spoza M , następnie krawędź z M , potem znowu spoza M itd.

Ponieważ ścieżka zaczyna się w wierzchołku wolnym z V'_1 i kończy się w wierzchołku wolnym z V'_2 , otrzymana ścieżka jest ścieżką powiększającą względem M .

$\Leftarrow$ W drugą stronę, załóżmy, że w grafie G istnieje ścieżka powiększająca względem M .

Taka ścieżka zaczyna się w wierzchołku wolnym z V_1 , kończy się w wierzchołku wolnym z V_2 , a jej krawędzie naprzemiennie nie należą i należą do M . Po zorientowaniu krawędzi zgodnie z definicją grafu G_M otrzymujemy ścieżkę skierowaną z V'_1 do V'_2 . Taka ścieżka zostanie znaleziona przez procedurę APS. $\square$

Złożoność czasowa procedury APS

- linie 1–2: wyznaczenie zbiorów wierzchołków wolnych zajmuje czas

$$O(|V|),$$

- linia 3: skonstruowanie grafu skierowanego G_M zajmuje czas

$$O(|E|),$$

- linia 4: znalezienie ścieżki w grafie G_M za pomocą BFS zajmuje czas

$$O(|E|),$$

- linie 5–8: zwrócenie wyniku zajmuje czas

$$O(1).$$

Zatem całkowita złożoność procedury APS wynosi

$$O(|E|).$$

5 Algorytm największego skojarzenia

Algorithm 2 MM(G) – Maximum Matching

Require: Graf dwudzielny $G = (V_1 \cup V_2, E)$

Ensure: Największe skojarzenie M w grafie G

```
1:  $M \leftarrow \emptyset$ 
2: repeat
3:    $P \leftarrow \text{APS}(G, M)$ 
4:   if  $P \neq \text{NULL}$  then
5:      $M \leftarrow M \triangle E(P)$ 
6:   end if
7: until  $P = \text{NULL}$ 
8: return  $M$ 
```

Twierdzenie 2. Algorytm MM(G) zwraca największe skojarzenie w grafie dwudzielnym G .

Dowód.

Algorytm rozpoczyna działanie od pustego skojarzenia $M = \emptyset$. W każdej iteracji szukana jest ścieżka powiększająca P względem aktualnego skojarzenia M .

Jeżeli taka ścieżka istnieje, wykonywana jest operacja

$$M \leftarrow M \triangle E(P).$$

Usuwa ona ze skojarzenia krawędzie należące do M i leżące na ścieżce P , a dodaje krawędzie ścieżki P , które wcześniej do M nie należały. Ponieważ P jest ścieżką powiększającą, po tej operacji rozmiar skojarzenia zwiększa się o jeden.

Algorytm kończy działanie wtedy, gdy $\text{APS}(G, M)$ zwraca NULL, czyli gdy nie istnieje ścieżka powiększająca względem M . Z twierdzenia Berge’a wynika wtedy, że M jest największym skojarzeniem w grafie G . $\square$

Złożoność czasowa algorytmu MM

- linia 1: inicjalizacja pustego skojarzenia zajmuje czas

$$O(1),$$

- pętla wykonuje się co najwyżej

$$\frac{|V|}{2}$$

razy, ponieważ po każdej udanej iteracji liczność skojarzenia zwiększa się o 1, a maksymalna liczność skojarzenia jest $\leq \frac{|V|}{2}$,

- linia 3: wywołanie procedury APS zajmuje czas

$$O(|E|),$$

- linia 5: aktualizacja skojarzenia

$$M \leftarrow M \triangle E(P)$$

zajmuje czas

$$O(|V|),$$

ponieważ długość ścieżki P jest nie większa niż $|V|$.

Jedna iteracja pętli zajmuje więc czas

$$O(|E|) + O(|V|) = O(|E|),$$

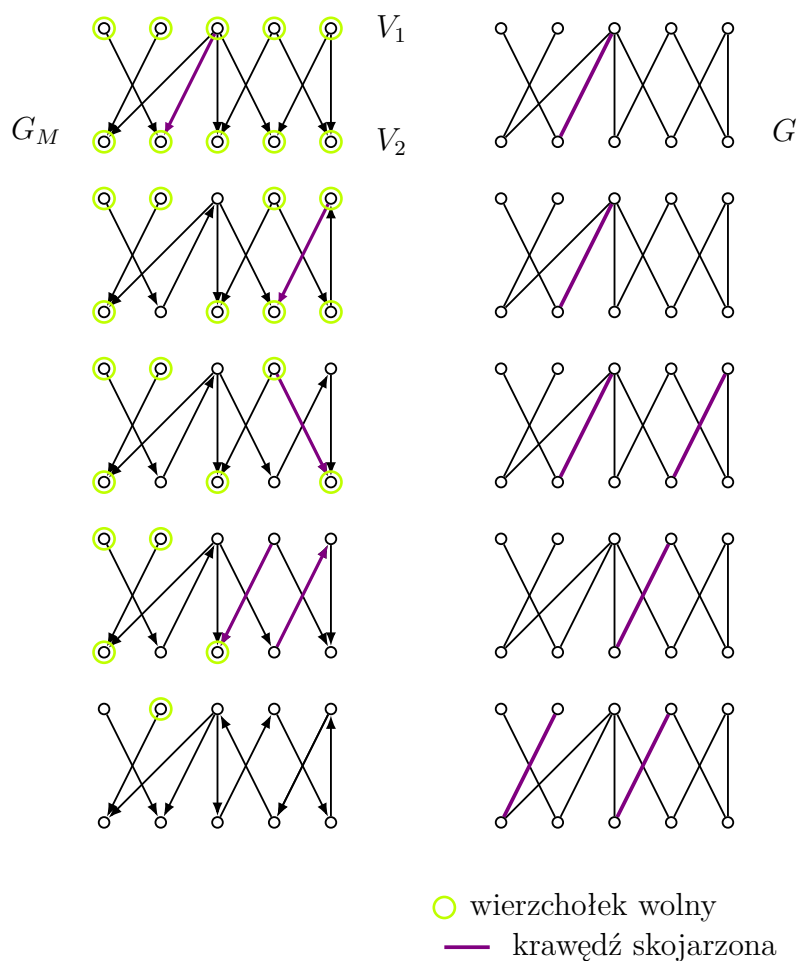
ponieważ w grafie bez izolowanych wierzchołków zachodzi $|V| = O(|E|)$.

Całkowita złożoność czasowa algorytmu MM wynosi

$$O(|V| \cdot |E|).$$

6 Przykład

Poniżej pokazano kolejne etapy działania algorytmu dla przykładowego grafu dwudzielnego G oraz odpowiadających mu grafów pomocniczych G_M .



Rysunek 5: Przykład: po lewej kolejne grafy pomocnicze G_M , po prawej odpowiadające grafy G z zaznaczonymi krawędziami skojarzenia.

